ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/354424369

D-Fence: A Flexible, Efficient, and Comprehensive Phishing Email Detection System

Conference Paper · September 2021

DOI: 10.1109/EuroSP51992.2021.00045

CITATIONS
0

READS
185

6 authors, including:



Fahim Abbasi
Massey University
11 PUBLICATIONS **80** CITATIONS
SEE PROFILE



Dinil Mon Divakaran
Trustwave
77 PUBLICATIONS **596** CITATIONS
SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project Business Email Compromise View project

Project Network security View project

All content following this page was uploaded by Fahim Abbasi on 07 September 2021.

The user has requested enhancement of the downloaded file.

D-Fence: A Flexible, Efficient, and Comprehensive Phishing Email Detection System

Jehyun Lee, Farren Tang, Pingxiao Ye, Fahim Abbasi, Phil Hay, Dinil Mon Divakaran
Trustwave
{*Jehyun.Lee, Farren.Tang, Pingxiao.Ye, FAbbasi, PHay, Dinil.Divakaran*}*@trustwave.com*

***Abstract*—Phishing continues to be a major security concern for organizations around the globe. Past works proposed classifiers to detect phishing emails; however many of them are based on rules, whereas others are typically standalone models focusing on one specific component of emails (say, URL strings). In this work, we take a different approach and propose a multi-modular and comprehensive phishing email detection system, called D-Fence. The different modules of D-Fence — structure module, text module, and URL module — detect phishing attempts in different components of an email. This allows D-Fence to cover larger attack surfaces while also offering flexible (model) configurations with reduced computational overhead. We carry out experiments on a large-scale real-world email dataset comprising mails from multiple enterprises. Our evaluations demonstrate the effectiveness of D-Fence in detecting phishing emails that do not have malicious intentions manifesting in all email components; D-Fence achieves a high recall of 0.99 at a low false-positive rate of 1 in 10K. Furthermore, we perform systematic evaluations to find and evaluate cost-efficient model configurations for D-Fence; the results reveal that D-Fence maintains high detection capability while bringing significant savings in computational time.**

***Index Terms*—phishing detection, phishing email, machine learning, deep learning**

1. Introduction

Phishing attacks have caused huge financial losses to users across different sectors [57]. Indeed, it is the largest attack faced by enterprises [68]. According to a recent report, 34% of all phishing attacks target enterprises [1]. Emails, in particular, constitute a prominent attack vector used to lure potential victims to phishing websites; around 1 in 100 emails is a phishing attack, and a staggering 25% bypass Office 365 default security [8]. Although there have been several proposals to detect phishing attacks, e.g., [27], [43], [70], there are still gaps to be addressed to keep up with the fast-evolving phishing email attack trends as well as the variety of phishing emails.

(i) Limited coverage: Most of the recent phishing detection solutions are based on specific email component — headers, URL links, text contents, etc. — in an email. For example, while [43] analyzes only URLs to detect phishing pages, [30] analyzes only the text contents in an email body. Such systems that rely on a specific email component are inherently limited by the attack surface they cover, and therefore prone to smarter attacks. An attacker, intentionally or not, may choose to use another component for luring the victim without exploiting the component used by the model (e.g., coaxing a recipient to reply, instead of providing a URL link). To have a good coverage of the phishing email ecosystem, a detection solution should fence much of the attack surface. To this end, we propose to detect phishing emails based on the most relevant components in an email; namely, email header, HTML structure of an email, text in natural language, and URLs embedded in an email (Sec. 3).

(ii) Limited models: The individual machine learning (ML) and deep learning (DL) models targeting at the detection of attacks on specific email components have two limitations. 1) Learning models being probabilistic in nature are prone to false predictions. When evaluated at low rates of false predictions, they miss detecting many phishing attempts. For example, the best performing model for URL-based phishing detection achieves a detection rate (recall) of less than 90% for a fixed false-positive rate (FPR) of 1 in 1000 mails (Sec. 6.4). 2) While existing phishing detection based on URL strings has demonstrated promising results [43], the same cannot be stated for the models on other email components. In particular, previous solutions for text-based content analysis, ranging from feature-engineering that captures semantics in the text [23], to the use of TF-IDF (term frequency–inverse document frequency) [70], to the application of DL based on simple word embeddings [30], have serious limitations. For instance, since methods such as TF-IDF, Bag-of-words, and Word2Vec, are highly dependent on the training corpus, the systems with these methods have weaknesses against contextual variances. This disadvantage is strongly pronounced when the datasets fall short of volume and diversity.

(iii) Limited focus on operational constraints: Learning-based solutions incur a cost (i.e., computational time) for extraction of features/embeddings, model building and prediction. And this cost depends on a few factors coming from operational constraints. For example, a security vendor may prefer to update a model every few days to keep up with the dynamics of evolving phishing attacks [50], with the millions of emails it collects, on a set of computing resources it has. Given that the data size used for training phishing models is usually huge (in millions of email samples), the (re-)training can take anywhere from a few hours to days, depending on the model and the computational resources available. For example, training time with the same dataset varies between 5 to 20 hours depending on the modeling configurations (Sec. 6.6). While research studies typically

present the model (configuration) that achieves the best detection capability, whether such configuration is practical and efficient has not been analyzed. Note that, in practice, the model building time also includes experimentation and validation of multiple models (configurations) before actual deployment. Prediction time, on the other hand, influences the resources to be provisioned to achieve a desired online mail delivery throughput (by a secure email gateway).

(iv) Limited dataset: To the best of our knowledge, with the exception of [33] (which deals with a related but different problem of lateral phishing detection; see Sec. 7), none of the past works have been evaluated on large-scale real-world enterprise datasets with diverse characteristics. Most works use publicly available datasets, which are modified significantly to minimize privacy leak [17], consequently reducing the diversity in different aspects of the emails (such as headers, for example). This issue is compounded by the fact that public datasets also have only a small number of phishing emails, due to the natural low rate of phishing emails [8], [18].

In this work, we propose a comprehensive phishing email detection system, named D-Fence[1], that is flexible, efficient, and effective. D-Fence takes a multi-modular approach employing multiple machine learning and deep learning classification models. As illustrated in Fig. 1, D-Fence consists of three modules (Sec. 4): i) structure module that uses a tree-based classifier for detecting phishing attacks on the header and HTML structure of an email, ii) text module that trains a classifier based on the embeddings generated by state-of-the-art language modeling technique BERT [20], and iii) URL module classifying URL strings using a deep-learning model. With multiple modules complementing each other in detection capability, D-Fence addresses the above two limitations. In addition, D-Fence allows flexible configurations and brings significant savings in computational time of building models and extracting features and embeddings, without compromising the effectiveness (in terms of accuracy).

Addressing the last limitation, we carry out comprehensive evaluations of D-Fence using a real-world dataset of more than 291K emails collected from multiple enterprises, in Sec. 5. This dataset, referred to as EES 2020, has important advantages in comparison to the public datasets used in most previous studies. (i) EES 2020 comprises emails from multiple enterprise customers, which is the most relevant dataset for phishing email detection systems. Such a real enterprise dataset has rarely been investigated previously, due to concerns on privacy and confidentiality. (ii) Phishing emails in EES 2020 are collected by multiple commercial tools deployed at customer networks and are large in number (Sec.5.1). Lastly, (iii) EES 2020 has emails with diverse properties representing real-world phishing characteristics. (see Tables 1 and 2). Importantly, there are emails that do not have all the components that many works usually assume. These real-world characteristics of phishing emails are what motivates us to propose a multi-modular comprehensive phishing detection solution.

The experimental evaluations (in Sec. 5) show that, D-Fence not only achieves high accuracy, but also detects phishing emails that are missed by high-accuracy yielding standalone classifiers. Furthermore, in Sec. 6, we carry out systematic experiments to evaluate D-Fence with reduced costly features for structural analysis models, limited word processing for text model, as well as reduced neural network complexity. Compared to the configuration yielding the best detection capability, the cost-efficient configurations reduce training and prediction time of structure module by 50% and text module by 80%, while retaining high recall at an FPR of 10^{-3}. In the URL module, a carefully configured deep learning model takes 40% less learning and prediction time. Overall, D-Fence demonstrates enhanced phishing detection performance under even lower FPRs, while still maintaining lower computational overhead — D-Fence achieves a recall of over **0.95** at an FPR of 10^{-5} in a cost-efficient configuration.

1. D-Fence expands as Deep-learning based flexible, efficient and comprehensive phishing email detection system.

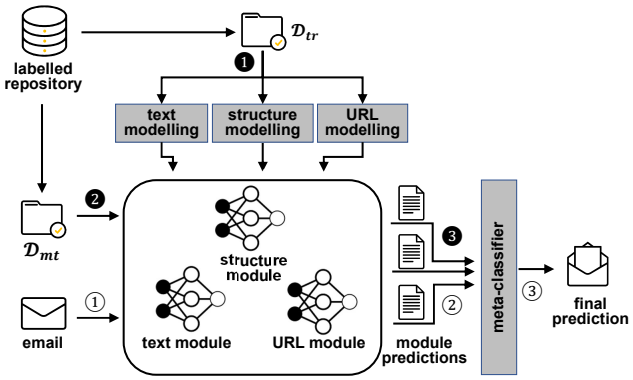


Figure 1: An overview of D-Fence. The filled and unfilled circles denote training and prediction phases, respectively.

2. Motivation

In this section, we describe our motivation on email component coverage of a phishing email classification system and define our threat model.

2.1. Component coverage

While adversaries modify certain parts of an email for creating a phishing email, security researchers utilize the same for detecting the attacks (via filtering rules and classifications); we refer to these parts of an email record as **email components**. The email headers and their values (e.g., sender's email address, message ID, presence of specific headers), text and HTML content sections, attachments, certain parts of the content sections (e.g., embedded URLs, sensitive words, and specific HTML tags), are examples of components that have been frequently utilized in previous studies. Fig. 12 in Appendix A.1 presents an example email highlighting these different components.

Attack surface and evasion of email components. An adversary sending a phishing email may leverage one or more components of an email (e.g., by embedding a URL to a phishing page) for malicious purposes. And the implementation of a malicious intention is strongly coupled with the specific email component exploited, in two-fold — technical availability and purpose. For example, if an adversary expects the recipient to click and access a link, a URL should come with a hyperlink tag in HTML format and/or a protocol indicator, e.g.,

`http://` or `https://`, to be automatically converted to a hyperlink by an email service application (such as, Gmail, Outlook, etc.). If an adversary wants a recipient to reply with a confidential document, the phishing email should have a convincing text that persuades the recipient. On the other hand, to evade a solution that analyses the reputation of the sender domain, an adversary has to forge the `from` field in the email header. Therefore, among the thousands of phishing emails encountered at an enterprise, there is clear diversity in the email components exploited by attackers.

Coverage in a phishing detection system. The capability of a phishing email classifier to obtain the prediction probability from a certain component with respect to all the possible email components can be referred to as the **coverage** of the system. A model that utilizes only one component (and achieves good accuracy), can only work well for those phishing attacks which have differentiating characteristics on that specific component (e.g., URL).

We argue that, i) the email components left behind by a detection system opens up a wider attack surface for an attacker to exploit, and ii) a classification system should be capable of classifying an email despite the absence of non-mandatory components.

2.2. Threat model

Phishing emails form the threat model in this work. We consider phishing emails that exploit different combinations of email components such as headers, HTML structure, text, and URLs. Attacks may bypass not only less accurate detection systems, but also those that do not utilize the components exploited by attackers.

We evaluate D-Fence using a large number of real-world phishing emails including those that have only a subset of the email components. In particular, the real-world email dataset EES 2020 (Sec. 5.1) includes mails that do not contain text, HTML, certain headers, URLs, and a combination of the mentioned scenarios.

In this work, we do not cover email attachments; however, the modularized architecture of D-Fence supports existing and future classifiers for attachment analysis to be easily adopted. While there are adversarial attacks that target specific models (e.g., [39], [44]), we consider enhancing individual models to be robust against such adversarial attacks as out of scope of our current work.

3. Background

In this section, we provide the background for the techniques adopted in D-Fence.

3.1. Email structure

We briefly introduce the email components that D-Fence utilizes. Headers in an email record are used by email applications (e.g., Outlook and Webmail services), SMTP servers, and third-party services between the end-to-end SMTP servers (e.g., email security solutions). Standards and proposals on these email headers [6], [53], [63], [64] define the format and usage of the headers, but not without restricting additional customized headers.

Many emails in the wild have multiple content sections as email conveys. The content sections are partitioned by section boundaries (e.g., `--00000...781811` in Fig. 12 in Appendix A.1), and each may have its own content type, character set, and encoding. We focus on `text/plain` and `text/html` sections that render the contents to a reader. `text/html` is generated when an email composer explicitly uses HTML tags and sets the email client to consider the tag as HTML, and implicitly when a composer uses any of styled texts or any enhanced content format like a hyperlink, an embedded image, etc. Fig. 12 in Appendix A.1 illustrates the implicitly generated `text/plain` and `text/html` sections. Since most of today's email services support HTML, a reader would likely see a HTML section including all the style formats and hyperlinks in an email. These automatically generated sections (supporting enhanced email contents and application compatibility) come with necessary headers, such as the definition of `boundary` ID, use of boundary ID, `Content-Type`, `Content-Transfer-Encoding`, etc.

3.2. Machine Learning techniques

ML techniques are employed for detecting phishing attacks due to their efficiency and accuracy [14], [73]. In many classification problems, the most time-consuming part of training and prediction is often due to the extraction of features from data, in particular, when tree-based models are used. Hence, features need to be defined carefully, keeping both efficiency (time for extraction) and effectiveness (accuracy) in mind. Finding the optimal set of features that are both efficient and effective is a hard problem, as this requires evaluating correlation among the given features (see [15], for example).

Since the performance of ML classifiers depends mainly on the features used, existing phishing classifiers focus on defining and engineering effective features that represent the syntax, semantic aspects (such as the existence of sensitive words), and other phishing characteristics. For example, in the case of URL-based models, some of the commonly used features are the occurrence of IP address in the domain name, the existence of a redirection URL, the use of rare top-level domain names, etc. Whereas for HTML-based models, the number of unique HTML tags, the use of JavaScript, and the existence of 1×1 pixel-size `iframe`, etc. are typically used as features. To detect phishing emails in particular, ML-based approaches use a number of features, e.g., a hyperlink to an unknown URL, existence of sensitive words in the contents, presence of executable attachment, etc. [32]. Once the features are extracted from the source (URL/HTML/email), they are provided as input to train an ML model (such as RandomForest). The trained model is subsequently used to predict the probability of a given sample data (URL/HTML/email) as being phishing.

3.3. Deep Learning techniques

While ML solutions generally rely on feature engineering to define features that differentiate the target classes (e.g., phishing and benign emails), DL has the advantage of automatically learning the correlations among

the features from a given large dataset. The state-of-the-art DL models are convolutional neural networks (CNN), recurrent neural networks (RNN). These, along with LSTM (long short-term memory, an extension of RNN), are the most widely used deep-learning techniques in recent classification studies. CNN is generally composed of multiple building blocks, such as convolution and pooling, and learns spatial relations of features iteratively through backpropagation [29]. Whereas, RNN learns sequential correlations as it propagates selective information from the previous state to the current state [66]. LSTM overcomes the vanishing gradient problem of RNN [61] with an additive gradient update structure [35].

To develop D-Fence, we adopt a combination of CNN and LSTM for modeling URLs embedded in emails. In Sec. 4.3, we describe how our URL classification model is organized with these neural networks and the neuron processing methods.

3.4. Natural language processing techniques

Recent advances in NLP has seen successful adoption of neural networks; in particular, these new approaches have been applied to information security topics as well [7], [77]. Since one of the key attack vectors in phishing emails is text in natural language, detecting phishing emails is an interesting application of language models. As we describe in Sec. 4.2, D-Fence uses pre-trained transformers, specifically BERT (bidirectional encoder representations from transformers) [20], for extracting contextual embeddings from readable texts in emails.

Text classification usually requires the conversion of text into a sequence of numerical feature vectors. Commonly used techniques for this conversion are TF-IDF [49]. TF-IDF is a sparse matrix where each word maps to a single value indicating its importance, which in turn depends on the word's frequency; however, this computation is carried out without capturing relationships with other words. Word2Vec, on the other hand, captures relationships from other words in a sentence. Yet, it still does not consider the context of an input sentence but only the context with the surrounding words on the training set [24]. Word2Vec shows its practicality in many NLP tasks [3]. However, Word2Vec has a critical drawback; it learns one context per word across all the sentences on a training set. Once a model is trained, a word always has the same embedding vector. Thus the same word appearing in different sentences, or appearing multiple times in a sentence (and thereby potentially having different contexts) would have the same embedding.

BERT is a state-of-the-art technique that encodes a sentence in a bidirectional manner. BERT learns relations between sentences and between tokens by jointly conditioning on both left-to-right and right-to-left directions. The key advantage of BERT and similar techniques (e.g., ELMo [56] and OpenAI GPT [59]) is contextual text embedding. BERT provides different word embedding values for each word based on the input sentences. This contextual text embedding enables the model to differentiate the meaning of sentences comprising of similar words. Not surprisingly, there is a cost for this capability; obtaining the context-specific values for every input sentence places more computational overhead than Word2Vec and TF-IDF.

Another characteristic of BERT is subword tokenization. BERT uses subwords (e.g., 'subword' tokenizes into 'sub' and 'word'), specifically, WordPiece [72] for its vectorization token, instead of a stem of a word which is widely used in English text models. The use of subwords as tokens is known to be effective against the burst of vocabulary size besides for stemming. When a very rarely used word appears in an input sentence, a subword-based model can represent the new word with known subwords instead of constructing a new vocabulary.

4. D-Fence: system design and development

We now present and develop our phishing email detection system, D-Fence. We design D-Fence to address the shortcomings of existing phishing email detection solutions as well as to meet operational demands. D-Fence enables large coverage by employing analysis modules targeting various email components. Moreover, D-Fence works without any external information sources which not only need to be up-to-date for being effective (e.g., reputation scores), but also requires a local repository to maintain historical data (e.g., previous sender list). In other words, D-Fence is designed as an in-line solution (e.g., at a secure email gateway), where each email is processed independently, without any external communications or database lookups.

Lastly, the flexible architecture of D-Fence also lowers the cost (in terms of computational time) for regular (re-)training of the models due to operational demands; e.g., updating the model with new batch of training data, enhancing D-Fence by adding new features or replacing with improved models, etc.

Fig. 1 presents the overall architecture of D-Fence. It consists of three independent analysis modules, namely, the structure module, the text module, and the URL module, as well as the meta-classifier which is dependent on the output of the three analysis modules. We describe them below. $\mathcal{D}_{tr}$ and $\mathcal{D}_{mt}$ are two (non-overlapping) partitions of our dataset (Sec. 5.1); while $\mathcal{D}_{tr}$ is used for training the three modules, $\mathcal{D}_{mt}$ is used for training the meta-classifier.

4.1. Structure analysis

This module analyzes the existence and values of headers and HTML content structure in an email record as illustrated in Fig. 2. These include the number of hyperlinks, unique domain names in HTML assets and linked URLs, the tree structure of Document Object Model, etc. Email header is a source of detecting abnormal email senders and crude quality of phishing emails that are distinct to benign emails. Benign emails tend to have diverse standard headers with multiple entries (e.g., stacked `Received` headers added by layered email infrastructure, and authenticated received chain headers [6]) as well as non-standard headers (e.g., `X-Google-Smtp-Source` by a Gmail SMTP server), since they are sent via authentic email service providers.

HTML contents tend to possess inherent characteristics of phishing emails since most of the technical aids for malicious functionalities are hardly supported without use of HTML contents [22]. An example mail where a

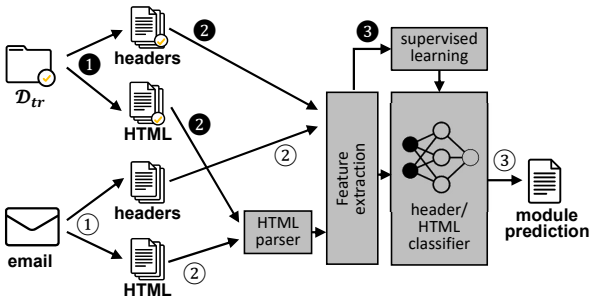


Figure 2: Training and testing of structure module

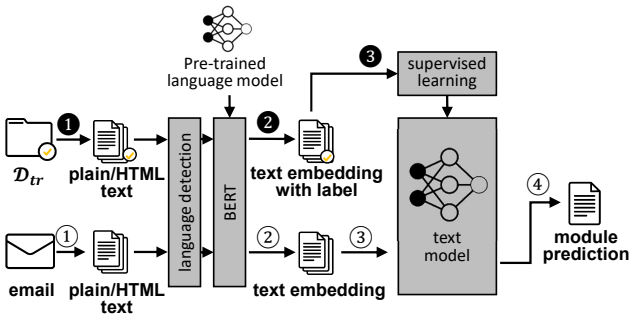


Figure 3: Training and testing of text module

hyperlink text points to an unexpected URL is illustrated in Fig. 12 in Appendix A.1.

In modeling these features, structure module uses a composition of traditional structural features from email header/HTML, inspired from previous studies, in particular, [2], [11], [22], [27]; besides we also introduce features based on our domain knowledge and empirical observation on real-world phishing emails., e.g., data type of message ID (decimal, hexadecimal, or character string), special characters in the section boundary IDs, etc. In Appendix A.5, we list the structural features.

Feature extraction and modeling. The structural features can be efficiently extracted with a single-pass scan of an email. The email scan covers the threaded emails, if an input email record includes the previous emails capturing conversation behavior between sender(s) and recipient(s). Specifically, this module extracts the standard email headers [53], [63], [64], and their values; it also extracts features from HTML content from the sections where the value of `Content-Type` is `text/html`. The readable texts and URLs detected while parsing an email structure are forwarded to the text module (in Sec. 4.2) and the URL module (in Sec. 4.3), respectively. For modeling these features, we use well-known ML algorithms, e.g., RandomForest and XGBoost.

Advantage on coverage. Since the basic header fields such as `from` and `to` are mandatory for sending an email, they are common to both phishing and benign emails. Furthermore, most benign emails have many more headers such as `subject`, `reply-to`, and `return-path`, which are often mandated by most email services. Coming to phishing emails, they are massively generated by an adversary, typically using a template and forging a fake identity of a sender; therefore, they are observed to have lower number of headers and also less diversity in the header values [31]. The absence of headers and their values make an email suspicious. This is in contrast to the case where a URL and an HTML section is absent, which may lead a detection system to conclude the mail is benign. Investigating HTML sections enables us to cover most of the phishing emails, since HTML is a necessary component for many phishing attack vectors [22]. According to our observation of the real-world enterprise dataset EES 2020, while 92.49% of the phishing emails have one or more `text/html` sections, only 77.43% of the benign emails have them (refer to Table 1).

4.2. Text analysis

The text module predicts how likely the sentences belong to a phishing email. Though modeling natural language content is a much heavier task compared to (modeling) the contents having structured syntax (e.g., email structure, headers, and HTML), texts are critical since they form a key component in deceiving victims. It is not surprising that a well-crafted and convincing text persuades a recipient to reply with sensitive information [16] or access a suspicious attachment or hyperlink.

Studies (e.g., [40], [41], [47]) have shown that natural language modeling techniques are capable of classifying texts effectively; and NLP models have also been useful in detecting phishing mails as well [25], [30], [58]. We utilize the neural network model trained on English language by Bidirectional Encoder Representations from Transformers (BERT) [20] for modeling the texts in phishing and benign emails. We extract the BERT embeddings from the texts on an email and use the embeddings as the features for a text classification model.

Text preprocessing and language detection. To obtain the text embeddings, we first extract the texts (❶ in Fig. 3) from the plain text section (where the `Content-type` is `text/plain`) and from the HTML section. Since BERT uses a pre-trained language model, the texts in different languages need to be modelled and investigated separately. To this end, we detect the language of the texts and ignore the emails which are not matched to the pre-trained language model, i.e., English in our experiments.

We filter out any non-text elements in the sentences, e.g., URLs, email addresses, numbers, special characters, etc., to remove noises during language detection, as well as during the feature extraction process. The ignored non-English emails will thus have a meaningless prediction value for the text analysis, and will eventually be classified with the predictions from the other two analysis modules.

Contextual text embedding extraction. BERT transforms sentences into multi-dimensional text embeddings (❷ in Fig. 3). Fig. 4 illustrates how we use BERT to extract features from a text input. The sentences extracted from an email first undergo a process called tokenization. BERT tokenizer splits sentences into subwords (introduced earlier in Sec. 3.4), to make a sequence of tokens for transformation. The token sequence consists of the subwords denoted as t_k in Fig. 4, including special tokens, i.e., `[CLS]`, `[SEP]`. The special tokens are automatically placed by the BERT tokenizer and share the token positions with t_k. Specifically, `[CLS]` indicating 'Class' always takes the first place in the token sequence, and the embedding vector for `[CLS]` is often used for text classification. `[SEP]` indicates sentence separation when there are multiple sentences within a token sequence.

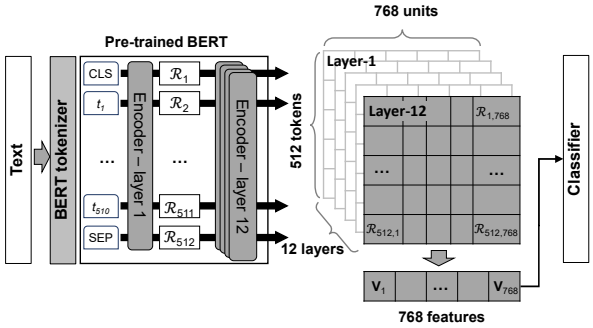


Figure 4: BERT embedding extraction flow in text module.

BERT transforms the full token sequence into a vector of numerical values per token. We denote by $\mathcal{R}$ a list of vectors, where $\mathcal{R}_i$ is the vector of numerical values representing the contextual embeddings for each corresponding token. This transformation is conducted through multiple-layered pre-trained encoders, and the length and the number of the output vectors from each layer are determined by the pre-training set. We use a BERT pre-trained with English Wikipedia and a book corpus named *bert-base* [38]. This bert-base has 12 encoder layers and 768 hidden units. When the tokens are fed as input into the bert-base, each token is transformed into a vector with 768 real numbers for each layer. Though there is no fixed method to select or aggregate these multi-dimensional embeddings, we use the 512 embedding vectors from the final layer (i.e., layer 12), and aggregate the vectors of the tokens into a single vector by taking the average. Algorithm 1 lists the steps involved in extracting the aggregated BERT embedding vector from text sentences. Once the input sentences $\mathbf{S}$ are preprocessed and tokenized, the special tokens and subwords form the token sequence $\mathbf{T}$. After BERT transforms $\mathbf{T}$ into 512 embedding vectors, i.e., $\mathcal{R}_{1,..,512}$, we additionally aggregate the output vectors $\mathcal{R}_i$'s into a single feature vector $\mathbf{V}$ by performing element-wise average across the 512 vectors. In Algorithm 1, with a slight abuse of notation, we let $\mathcal{R}_{i,j}$ to denote the jth element of the vector $\mathcal{R}_i$. We train a supervised classifier with this 768 feature vector (❸ in Fig. 3). During operation (i.e., the testing phase), the features are extracted from an email through the same process (① and ②), and provided as input to the text classification model for prediction (③). The prediction probability from the text classifier is used as a feature for the meta-classifier (as illustrated later in Fig. 6.

Advantage over message-centric phishing. Text analysis provides classification capability against phishing emails having malicious intent manifested in text. For example, a spear-phishing email carefully crafted by an adversary and sent through a genuine email service hardly has differentiating characteristics in its email header and HTML contents [27]. In particular, an email serving as a social-engineering attack achieves its goal by deceiving and convincing an email reader to follow instruction.

For example, a phishing email impersonating a secondary email address of one's boss may lead a victim to leak confidential information to the attacker. Such attacks need not necessarily have a hyperlink or an attachment file; instead, text alone is sufficient to coax the victim to respond. This type of phishing email naturally has a higher possibility of containing sensitive words and sentences

Algorithm 1 BERT text embedding vector extraction

Input: $\mathbf{S}$; ▷ $\mathbf{S}$: sentences
$\mathbf{W} \leftarrow$ TextPreprocess($\mathbf{S}$) ▷ $\mathbf{W}$: words
$\mathbf{T} \leftarrow$ BERTtokenizer($\mathbf{W}$) ▷ $\mathbf{T}$: token sequence
$\mathcal{R} \leftarrow$ BERT($\mathbf{T}$) ▷ $\mathcal{R}$: a list of vectors
for $j \leftarrow [1, 768]$ **do**
  $\mathbf{V}_j \leftarrow$ Average($\mathcal{R}_{1,j}, ..., \mathcal{R}_{512,j}$)
end for
return $\mathbf{V}$ ▷ Vector of word embeddings

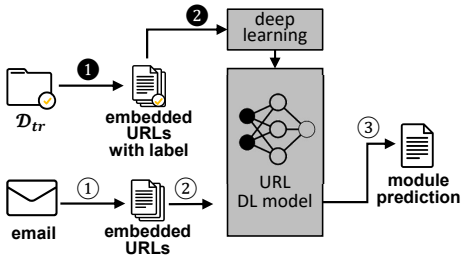


Figure 5: Training and testing of URL module

that our text analysis module aims to detect, while other analysis modules may predict a lower phishing probability.

4.3. URL analysis

Embedded URLs in an email are an immediate channel to lead a victim to a subsequent stage in the attack, such as a phishing webpage, malware infection, or web exploit. A URL can be placed in the middle of the text as well as a hyperlink in an HTML content. Lexical and structural aspects of phishing URLs [13], [42], as well as the reputation of URLs and their domain names [54], have been used as classification features in previous studies. Recent research works also demonstrate promising accuracy in detecting phishing URLs solely based on URL strings (with no external reputation scores) by employing deep learning techniques [10], [43].

The URL module adopts a deep learning approach to classify URLs in phishing emails without external reputation sources. We use a combined architecture of CNN and LSTM, which, based on our empirical observation, has the most steady performance compared to standalone CNN or LSTM architectures (Sec. 6.1). This neural network is designed to capture the differentiating aspects of phishing URLs on a character/word level, such as a random string, intentional typos and words with no semantic meaning. Recent studies have also shown the effectiveness of such a combined architecture [25], [74]. Since deep learning layers include the layer for pre-processing of URL strings into character embeddings and the layer for prediction (Sec. 3.3), URL module has the simplest structure as illustrated in Fig. 5.

URL modeling with DL. The URL strings extracted from `text/plain` and `text/html` contents are used for training and testing after encoding a character to an integer value. We use a CNN-LSTM architecture for this module, wherein the convolution layer extracts local features, while LSTM focuses on learning the long-term sequential dependency from the encoded characters. For pooling between the convolution layers, we use MaxPooling. We place a dense layer with multiple dense units after LSTM

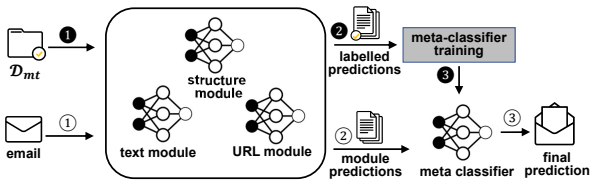


Figure 6: Training and testing process of meta-classifier

to aggregate the outputs from LSTM units (e.g., from 512 LSTM units to 256 dense units), and use another dense layer with a single unit to make a single output value representing the prediction probability. We give an illustration of this CNN-LSTM architecture in Fig. 13 in Appendix A.2. The network returns predicted probability for each individual URL. For obtaining a prediction value for an email with multiple URLs, we take the highest prediction probability of all embedded URLs in the email. The rationale for this is, presence of one phishing URL is sufficient to classify a mail as a phishing attempt.

Advantage in classification efficiency. In EES 2020 dataset, URLs appear in the majority of the emails, i.e., 78.60% of benign and 93.85% of phishing. Since URL extraction is carried out during the single-pass scan of emails performed to extract features for the other two modules, it does not introduce additional overhead.

4.4. Meta-classifier

Meta-classifier aggregates the prediction of individual modules into a single prediction probability for each email. Each prediction probability from a classification module has a value between 0.0 to 1.0 for meaningful input. Otherwise, the prediction has an alternative irrelevant value, e.g., -1, when the component for a module is missing (say, when there is no URL in an email). Taking the prediction probabilities from the individual independent modules as a feature vector, a supervised learning algorithm for the meta-classifier builds a classification model for predicting phishing emails.

As mentioned earlier, we train the meta-classifier using the partition $\mathcal{D}_{mt}$ of the dataset (❶ in Fig. 6). This partition $\mathcal{D}_{mt}$ is used for training only the meta-classifier (and in particular, not for training the independent analysis modules); thereby we avoid any potential over-fitting. To train the meta-classifier, we first employ the three modules to predict on $\mathcal{D}_{mt}$; then the labelled predictions from these modules (❷ in Fig. 6) are provided as features for training and building the model corresponding to the meta-classifier (❸ in Fig. 6).

We employ a simple supervised algorithm to learn the three features, i.e., one prediction per module. Although there are no significant differences among well-known classifiers we experimented with— Decision tree, RandomForest, and XGBoost — in terms of accuracy and computational cost, we use XGBoost for the meta-classifier as it is more flexible in configuring thresholds for extremely low false-positive rates (e.g., 10^{-5}).

5. Performance Evaluation

Here we evaluate D-Fence for its detection capability. In the next section, we evaluate the flexibility of D-Fence in maintaining a high detection rate when computational overhead of the independent modules is reduced.

TABLE 1: Ratio of email samples, across different components in our EES 2020 dataset. Note, Text could come from `text/plain`, `text/html` or both.

Content	Source	Label	No. of samples	Ratio
Text	Any	Benign	212200	94.67%
		Phishing	64587	95.59%
	text/plain	Benign	188261	83.99%
		Phishing	12039	17.82%
	text/html	Benign	136084	60.71%
		Phishing	59016	87.35%
HTML	text/html	Benign	173542	77.43%
		Phishing	62488	92.49%
URL	All	Benign	197087	87.93%
		Phishing	67559	99.99%
Total	All	Benign	224137	100%
		Phishing	67565	100%
		All	291702	

5.1. Dataset

Enterprise email samples (EES 2020). To evaluate our system, we collected 377K of emails during the period Jan. 2018 to Sep. 2020 from a reputed commercial cyber security company. To ensure that the samples are unique and non-overlapping between training and testing sets, we removed the duplicate samples which have, both, identical `Message-ID` and identical hashes, for `text/plain` and `text/html` sections. Emails having identical `Message-ID` but different hashes of `text/plain` or `text/html` are considered as distinct emails. Since reuse of Message-ID and small alteration in text or other components are also characteristics of phishing emails, we retain those samples. We use the remaining 292K emails for our evaluations. Table 1 summarizes EES 2020 dataset with respect to the email components we have described.

The phishing emails in this dataset come from multiple enterprise customers of the security company and were detected by multiple commercial tools deployed on secure email gateways at the customer networks. No information about the customers and the detection tools is made available to the authors, nor is it used by our proposed D-Fence system. Only authorized employees of the security company (including some of the co-authors) have access to this particular email dataset; and therefore, as described in the next section, restrictions were put in place to ensure the models are executed on specific machines within the company's network, complying with the company's security policies. In other words, data in any form did not leave the company's network.

Ground truth: While it is improbable to ascertain that the ground truth of the dataset is 100% accurate, it is worth noting that the false positives due to the multiple detection tools deployed are flagged out by the customers, and consequently corrected by the security company. Similarly, phishing mails not detected by the deployed tools are often reported by customers. We confirm that our dataset indeed has such corrected samples. Thus, with the recipients themselves becoming the final validator, we

TABLE 2: Statistics of EES 2020 dataset

Statistics	Label	Numbers
No. of organizations	All	∼200
Unique Recpt Email addresses	Benign Phishing	116K 9K
Unique Sender Email addresses	Benign Phishing	76K 15K

TABLE 3: List of public datasets

Test dataset	Label	No. of samples
Enron [17]	Benign	517401
SpamAssassin [28]	Benign Spam	6951 3793
Nazario [52]	Phishing	4045

believe this additional layer of scrutiny makes ground-truth reliable. This also means that, a model trained on this dataset is also likely to learn more and perform better than the commercial tools.

Diversity: Due to the confidentiality of the dataset, unfortunately, we cannot disclose the dataset and many related sensitive information. However, since the diversity of a dataset has some influence on the evaluation results, in Table 2 we share several statistics that support the diversity of EES 2020 dataset.

Public email dataset. We also test our system with several labelled email datasets available in the public domain [17], [28], [52]. Table 3 gives the number of samples in these public datasets. We train and test with 524,352 benign and 7,838 spam/phishing samples, using the same experiment configuration as for the EES 2020 dataset. We use this public datasaet for evaluating the accuracy of the individual modules and D-Fence in Sec. 5.5.

5.2. Experimental environment

The experimental environment has multiple layers of security due to the high confidentiality of emails. In particular, two machines with and without Graphics Processing Unit (GPU) were used for evaluations. Both machines operate within the security company's network. Since headers and texts are the most sensitive parts of emails, the corresponding analysis modules, namely structure module, and text module, are run on a machine (without GPU) with the strictest security policies. This machine runs on Intel CPU (8 physical 2.4 GHz cores) and has 32GB RAM. The URLs, on the other hand, were extracted into another machine with Tesla P4 GPU having 8GB RAM and running on a Ubuntu 16.04 OS; the URL module was tested on this machine. Our prototype system is implemented with Python 3.8 and Tensorflow 1.9.0 to be compatible with GPU driver and CUDA versions.

5.3. Performance metrics

True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) are the fundamental metrics for evaluating classifiers. As phishing email classification is an imbalanced class problem in practice and in our datasets as well, precision and recall are the appropriate metrics for evaluating the corresponding models [19]. Recall ($\frac{\text{TP}}{\text{TP+FN}}$) is the fraction of all phishing emails detected by a classifier. Precision ($\frac{\text{TP}}{\text{TP+FP}}$) gives a measure of how correct the phishing predictions are. For our evaluations, we also use the area under precision-recall curve (AUPRC) as a proxy for model accuracy.

In addition, we evaluate the models at low false-positive rates (FPR), since low FPR is an important criterion from an operational perspective. Every mail falsely predicted as phishing (i.e., false positive) does not reach the (focus) mailbox of users/customers, leading to customer dissatisfaction and costly hours of manual investigation, consequently affecting the trust in the solution. To give an example, assuming an enterprise receives thousands of emails per day, an operational constraint might be to have an FPR of not more than 1 in 1,000 emails (10^{-3}). Therefore, we fix FPR at low values and analyze the recall (TPR) that can be achieved by the models; in particular, we emphasize this metric in Sec. 6, where we carry out experiments to find practical cost-effective configurations of D-Fence.

5.4. Module configurations

The configuration of the ML and DL models in this section are set so as to achieve the best accuracy based on experiments in Sec. 6, wherein we vary the configurations to analyze the trade-off between computational overhead and accuracy. We use all defined 63 features (the full list is provided in Appendix A.5) for structure module, to achieve the best-accuracy. For text module, we use *bert-base* model described in Sec. 4.2 with 600 maximum input words. The bert-base uses only up to 512 tokens after removing unnecessary words and tokenizing the words (using BERT tokenizer). In URL module, we use CNN-LSTM architecture illustrated in Sec. 4.3. In particular, the neural network uses 256 Conv1D filters, 4 MaxPooling window size, 512 LSTM units, 0.1 Dropout rate, and 256 Dense units, and is trained in 4 epochs; this hyper-parameter configuration results in the best accuracy. We use *sigmoid* activation function at the last two dense layers. For the classifier in the structure module and text module we use RandomForest; and for the meta-classifier, based on our experiments, we choose XGBoost. The values of the hyper-parameters of these models are the default ones in the Python scikit-learn library [55].

5.5. Classification accuracy

We now compare the performance of the independent analysis modules and the meta-classifier. We partition the EES 2020 dataset into three sets, i.e., $\mathcal{D}_{tr}$, $\mathcal{D}_{mt}$ and $\mathcal{D}_{ts}$, in 40:20:40 ratio. To recall, $\mathcal{D}_{tr}$ is the training set for the analysis models. $\mathcal{D}_{mt}$ is training set for the meta-classifier as described in Sec. 4. $\mathcal{D}_{ts}$ is the test set for evaluating the end-to-end system. Therefore, the analysis modules are evaluated with a dataset of 40:40 training/test ratio ($\mathcal{D}_{tr}$:$\mathcal{D}_{ts}$), and the meta-classifier with 20:40 training/test ratio ($\mathcal{D}_{mt}$:$\mathcal{D}_{ts}$). In addition, to ensure the generality of the

measurements, we perform 10-fold cross-validation on the dataset obtained by merging $\mathcal{D}_{tr}$ and $\mathcal{D}_{ts}$ partitions. For the cross-validations, where there is no $\mathcal{D}_{mt}$ partition, we use the meta-classifier that is trained using the 40:20:40 ratio split experiment. We also use cross-validation for performance comparisons with the baseline methods.

On EES 2020 dataset. With 10-fold cross-validation on EES 2020 dataset, the meta-classifier achieves an AUPRC of 0.9997 with 0.0001 of standard deviation (σ), while structure module, text module and URL module, achieve AUPRC (σ) values of 0.9993 (0.0003), 0.9757 (0.0039), and 0.9851 (0.0031), respectively. While AUPRC values may convey that all three modules have similar detection capabilities, at a fixed FPR of 10^{-3}, the recall for structure, text, and URL modules are 0.9933 (0.0017), 0.7796 (0.0038), and 0.7648 (0.0353), respectively. Whereas, at the same FPR, D-Fence achieves a high recall of 0.9935 (0.0013).

Precision-recall curves and FP analysis presented are based on 40:20:40 ratio experiment on the test set partition $\mathcal{D}_{ts}$. Fig. 7 plots the precision-recall curve depicting the detection performance of the individual modules as well as D-Fence, evaluated on the test set $\mathcal{D}_{ts}$ of EES 2020 dataset. The individual modules achieve recall of 0.9878 (structure), 0.6182 (text), 0.8806 (URL), and 0.9932 (D-Fence) at 10^{-3} FPR. The gap between the best performing individual module — structure module — and D-Fence increases further to more than 0.04 at an even lower FPR of 10^{-4}; this translates to missing detection of additional 1K phishing emails (in EES 2020 dataset) by the structure module. While the high detection capability of the structure module can be attributed to a large number of naive phishing emails that are easily distinguishable by only email headers, it is worth noting (in Table 1) that, nearly 7.5% of phishing emails in EES 2020 do not have HTML section. This is where the other modules contribute with their detection capabilities.

On public dataset. We also evaluate the modules and D-Fence on publicly available datasets (see Table 3). With 10-fold cross-validation, D-Fence achieves 0.9999 of AUPRC (standard deviation, $\sigma = 0.0001$) and a recall of 1.0 at 10^{-3} FPR. Individual structure, text and URL modules achieve AUPRC (standard deviation, σ) of 0.9996 (0.0005), 0.9019 (0.0109), 0.9659 (0.0065); whereas recall values at 10^{-3} FPR are 0.9990 (0.0011), 0.8258 (0.0155), 0.7971 (0.0276), respectively.

In the experiment with 40:20:40 ratio on the test partition $\mathcal{D}_{ts}$, at 10^{-3} FPR, we observe slightly lower recall values of 0.9915 (structure), 0.7954 (text), 0.6897 (URL), and 0.9994 (D-Fence), due to the decreased training set size. The results are as given in Fig. 8.

Although the public dataset has a largely different composition of benign/phishing samples in comparison to the enterprise dataset EES 2020, only the text module demonstrates a different performance. Specifically, in spite of a higher recall of 0.8258 (at an FPR 10^{-3}), compared to 0.7796 in EES 2020 dataset, the text module in the public dataset shows much less AUPRC (0.9019) than the text module in EES 2020 dataset (0.9757). We observe sudden fall of precision at high recalls, as can be noticed in Fig. 8. due to the increase of false positives. The fall can be due to the extremely imbalanced benign/phishing ratio (98.5: 1.5) in the public dataset. A small fraction of false

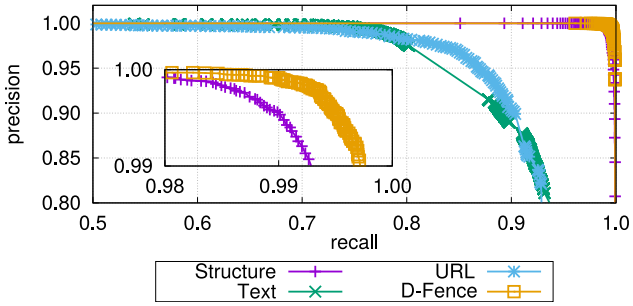


Figure 7: Performance of D-Fence and individual modules on EES 2020 (refer Table 1). Note, the plotting ranges are zoomed in for better visibility.

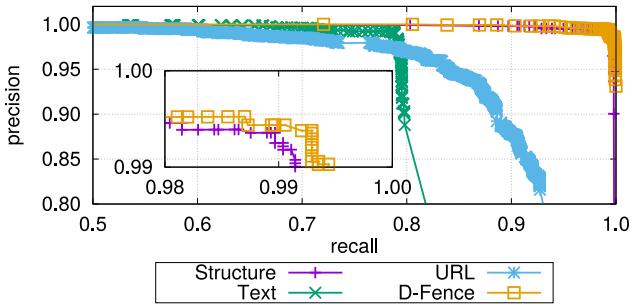


Figure 8: Performance of D-Fence and individual modules on public dataset (refer Table 3).

positives from a huge benign set can significantly decrease the precision; for example, with only 1.3% of falsely predicted benign samples, the precision would drop to 0.5 at 80% recall. Besides, when it comes to text analysis, it is easier for the email texts to have borderline cases, i.e., phishing-like benign and benign-like phishing emails, in comparison to email structures and URLs.

Performance comparison with baselines.

Next we compare the accuracy of D-Fence with the state-of-the-art studies. Since the structural features have been widely used in many ML-based studies for phishing/spam detection, we take features from multiple existing works (noted in Sec. 4.1) to define as a baseline for structure analysis. We propose several new features, and study the improvement due these by comparing it with the legacy features; both feature sets are trained with RandomForest. The list of legacy and new features is given in Table 10 in Appendix A.5. For text analysis, we compare D-Fence with DeepAnti-PhishNet that uses Word2Vec and LSTM for email text modelling [58]. For URL analysis, we consider CNN-LSTM model of [74] as the baseline. We implement the baseline systems following their key architecture and use hyper-parameters corresponding to the best accuracy on EES 2020 dataset.

Table 4 shows that D-Fence outperforms the existing systems that deal with single components. AUPRC and recall at 10^{-3} FPR are measured with 10-fold cross validation, and the corresponding standard deviations are given in parentheses. In particular, the Word2Vec vectorization used in [58] shows significant performance gap at a low FPR of 10^{-3}, in comparison to our approach using BERT. According to our further experiments discussed below — refer to Table 6, BERT with LSTM model achieves 0.6896 recall at 10^{-3} FPR, much higher than that achieved with

TABLE 4: Performance comparison with baseline systems in EES 2020 dataset. Combined features extends legacy features with our newly proposed features (refer Table 10).

	System	AUPRC (σ)	Recall (σ)
Baselines	Legacy structure features+RF	0.9985 (0.0002)	0.9663 (0.0051)
	Text Word2Vec+LSTM [58]	0.8313 (0.0074)	0.1365 (0.0023)
	URL CNN-LSTM [74]	0.9851 (0.0031)	0.7648 (0.0353)
Our proposals	Combined structure features+RF	0.9993 (0.0003)	0.9933 (0.0017)
	Text BERT+RF	0.9757 (0.0039)	0.7796 (0.0038)
	D-Fence	0.9997 (0.0001)	0.9935 (0.0013)

TABLE 5: Performance comparisons of different models for structure module. Recall is measured at FPR of 10^{-3}. Train time denotes the time taken (in seconds) for training on $\mathcal{D}_{tr}$ of EES 2020, and test time is the average prediction time per email (in milliseconds).

Model	AUPRC	Recall	Train (s)	Test (ms)
RandomForest	0.9993	0.9933	5	0.01
XGBoost	0.9994	0.9884	10	0.01
SVM (SVC)	0.9969	0.9618	919	0.55
Naive Bayes	0.8940	0.0	2	0.01

Word2Vec, but closer to the BERT with RandomForest model (0.7796). Finally, we adopt the combined structural features (including new ones that lead to improved performance in Table 4) and the CNN-LSTM based URL model into D-Fence, after evaluating with various hyper-parameter values in Sec. 6.1.

5.6. False predictions analysis

We manually investigate the false positive and false negative cases from the individual modules and the meta-classifier. Specifically, we select the top ten false positives (negatives) with the highest (lowest) prediction probability. Broadly, each module has its own weaknesses based on the differentiating features/characteristics their models capture. For example, the DL-based model in the URL module is after-all only modelling the strings of two classes of input data. Often, such models capture the overwhelming aspects of each class; for example, presence of randomly appearing characters is common in a large number of naive phishing URLs. But, it is perfectly possible for an attacker to craft a domain name and path that appears meaningful and non-random. We present such case studies pertaining to each individual module in Appendix B.1. By combining multiple models, D-Fence is able to make up for the performance losses of individual modules.

6. Flexible Model Optimization

In this section, we analyze the trade-off computational cost (in terms of time for training and prediction) and detection capability. All evaluations in this section are based on the EES 2020 dataset. Time for extracting features, text embeddings, training models and predictions are all important considerations for deployment. Learning-based models need to be updated frequently and regularly to capture the characteristics of fast-evolving attacks. Given a specific set of computational resources (computing power, memory size, etc.), large datasets for training, the training time decides how fast a newly (re-)trained model is available for deployment.

Prediction time determines the additional latency a mail has to wait before being delivered. Since mails are delivered asynchronously with delays often in the order of 10s of seconds to minutes, an additional delay of a few seconds can be considered as tolerable. Yet, the time taken by a model to predict is important, as it decides the throughput of the system, and resources that need to allocate for the processing mails in-line at a desired throughput. We analyze the trade-off of each module separately first, and then analyze the meta-classifier.

6.1. Model selection in individual modules

We evaluate the modeling options for each module, before proceeding to find the cost-efficient configurations of each module.

Models for structure module. We experimentally compare various ML algorithms in terms of accuracy and computational cost. For this purpose, we use the training set $\mathcal{D}_{tr}$ and the test set $\mathcal{D}_{ts}$ (two partitions from EES 2020 dataset). We train and test four well-known ML algorithms using 63 features; the results are as shown in Table 5 (the recall at different values of FPR are provided in Fig. 14 of Appendix A.3). As can be observed, the support vector machine (SVM) classifier incurs high processing time, while achieving high AUPRC. Naive Bayes classifier has the least detection capability; while the AUPRC looks good, the recall at an FPR of 10^{-3} is close to 0. On the other hand, both RandomForest and XGBoost, achieve high AUPRC and recall (at FPR of 10^{-3}) with a low computational cost. We also note that, the computational time for model training and prediction is, in particular for tree-based models, relatively small in comparison to feature extraction. Therefore, we analyse the accuracy that can be attained with reduced features in Sec. 6.2; for this purpose we select RandomForest and XGBoost for structure module.

Models for text embedding and classification. As described in Sec. 4.2, the text module uses two models: one for extracting text embeddings and another for text classification. For extracting the text embeddings, we consider Word2Vec in addition to BERT. For text classification, note that the features are the text embeddings. Since the number of features is more than 700 (in text input), we not only evaluate tree-based ML models but also DL models for classification. The comparison results between Word2Vec and BERT are presented in Table 6. In this set of experiments, carried out on a machine with GPU support processing (Sec. 5.2), we use deep learning (CNN and LSTM) for eventual classification of text. The results confirm that BERT outperforms Word2Vec in detecting phishing texts. The reduced feature dimension by our aggregation described in Sec. 4.2 leads to much less training and prediction time while maintaining higher accuracy than Word2Vec.

We carry out another set of experiments to evaluate phishing text classification using traditional machine learning algorithms. As seen in Table 7, RandomForest and XGBoost achieve much higher detection capability

TABLE 6: Performance comparisons of different embedding methods and DL models for text classification. Metrics definitions follow Table 5.

Method	AUPRC	Recall	Train (s)	Test (ms)
W2V + CNN	0.8051	0.1348	149	0.22
W2V + LSTM	0.8313	0.1365	8131	15.33
BERT + CNN	0.8822	0.5763	27	0.05
BERT + LSTM	0.8973	0.6896	50	0.08

TABLE 7: Performance comparisons of different text classification models in text module; all use BERT embeddings as features. Metrics definitions follow Table 5.

Model (BERT+)	AUPRC	Recall	Train (s)	Test (ms)
RandomForest	0.9757	0.7796	61	0.01
XGBoost	0.9746	0.6995	560	0.02
SVM (SVC)	0.8310	0.0776	48392	8.44
Naive Bayes	0.7353	0.0	3	0.02

than the DL methods in Table 6; therefore we use these ML models for further performance analysis. Note, time measurement across Table 6 and Table 7 are not comparable since the ML models in our experiments do not utilize GPU. Fig. 15 of Appendix A.3 presents recall at multiple FPRs.

Deep learning models for URL module. We evaluate several DL techniques for URL string modeling, based on detection capability and training/testing time overhead. We train and test CNN, LSTM, and CNN-LSTM architectures with the same numbers of Conv1D units, LSTM units, MaxPooling windows size and Epoch (if an architecture has the corresponding layer). The experiments are conducted in the server with GPU described above (Sec. 5.2). Table 8 is the summary of our experimental results (further results are presented in Fig. 16 of Appendix A.3). CNN has the least processing time, in comparison to the other two architectures (having an LSTM layer). But, CNN-LSTM has the best detection capability; in particular, it has much higher recall at 10^{-3} FPR, than both CNN and LSTM. LSTM shows less accuracy in spite of significant computational overhead. Based on this observation, in the following, we use CNN and CNN-LSTM architectures in analyzing the cost-efficient configurations for D-Fence.

6.2. Feature set reduction in structure module

Next we analyze the accuracy of the structure module when features are reduced. We group features into categories, such that the cost of extraction of features in one category is the same for all the features in that category. For example, email header parsing leads to one category of features with the same cost; similarly, the more expensive HTML DOM processing extracts another set of features, all of which have the same cost. Therefore, generally, it hardly makes any difference if we remove one feature or multiple features from one category. We evaluate the model in structure module with all possible combinations of the feature categories; specifically, we define 10 feature categories. Therefore, there are 2^{10} combinations of feature categories we experiment with.

We measure feature extraction time, training time and prediction time. For different feature categories, we compute the recall (TPR) at a fixed FPR of 10^{-3}. Fig. 9 plots the recall v/s the training time for EES 2020. The X-axis is scaled such that, 1.0 denotes the longest prediction time when all 63 features are used for model building. Thus, 0.5 training time indicates a reduction of feature extraction time and model building time by half. Since the major computational part of training and prediction times are due to feature extraction, we observe a non-distinguishable plot for recall with varying prediction times, and hence do not present it here.

TABLE 8: Performance comparisons of different DL models in URL module. Metrics definitions follow Table 5.

Architecture	AUPRC	Recall	Train (sec)	Test (ms)
CNN	0.9406	0.5775	302	0.76
LSTM	0.9149	0.5787	7728	14.41
CNN-LSTM	0.9851	0.7648	4247	7.85



Figure 9: Structure module: Recall (at fixed FPR of 10^{-3}) v/s training time, for various combinations of features. Training time includes the time to extract features.

Each point on Fig. 9 corresponds to a unique combination of feature categories; thus there are 1024 points. By comparing the data points on a horizontal line (Y-axis), we can easily find the cost-efficient configurations. For example, while the model with the full feature set has a recall of 0.9878 (at 10^{-3} FPR), a model with a reduced feature set obtained by excluding `CSS` features and `DOM` features, achieves a similar recall of 0.9877 but with 15% less prediction time.

Not surprisingly, not all feature reduction results in the same benefits. While the model without `Section count` features and `JavaScript` features saves only 2% of prediction time with a loss of 0.001 in recall, another model achieves same recall and 12% time reduction in prediction time by omitting `JavaScript` features and `HTML links` features. Similarly, there also costly features contributing to the model's performance. For example, the model with the reduced feature set obtained by omitting the four costly feature categories in above two examples, (i.e., `JavaScript`, `HTML links`, `DOM`, and `CSS` features), achieves a recall of 0.95 (0.03 less than the full feature set) at 10^{-3} FPR. However, at an FPR of 10^{-5}, the recall with this reduced feature set drops to 0.75 in contrast to the full feature set that achieves 0.92 recall.

6.3. Shorter sentences for text module

Since text module uses BERT embeddings as the features for classification, the time for extracting BERT

TABLE 9: Performance differences in terms of recall at fixed FPR of 10^{-3} (Δr), relative training time (Δt), and relative prediction time (Δp) for different configurations.

Layer	Config.	Δr	Δt (%)	Δp (%)
Conv1D	128 → 256	0.0326	8.24	7.83
Max Pooling	4 → 8	0.0125	**-41.14**	**-39.57**
LSTM	256 → 512	-0.0284	4.13	1.22
Dense	64 → 128	0.0010	0.20	0.33
Epoch	1 → 4	**0.1592**	**261.21**	0.04

embeddings contributes to both training and operational overhead. According to our empirical analysis, for the BERT embedding process, the extraction time increases proportionally with the numbers of input words as reported in [21]. Yet, increased number of words does not lead to an increase in the classification performance beyond a point. For example, the increase in recall at 10^{-3} FPR, between a scenario with 100 words (0.7467) and another with 500 words (0.7484) as input, is negligible; whereas, the extraction time increases by around five times, from 124ms/email for 100 words to 577ms/email for 500 words, on the test environment in Sec. 5.2.

6.4. Optimizing URL module hyper-parameters

Building a neural network model in URL module has high computational overhead due to the complexity of the network and the layers. But a more complex network with more nodes and layers does not always lead to increase in the accuracy of prediction. Through experiments, we analyse the positive and negative effects of the hyper-parameters. Based on our CNN-LSTM architecture, we build thirty-two models by changing five hyper-parameters in each layer; the configuration changes are given in the second column of Table 9.

Hyper-parameter configurations. While there are numerous options for different hyper-parameters, we focus on the most dominant attributes for each neural network layer, and define multiple configurations. A configuration is represented as a tuple of five hyper-parameters with two options for each: 128 and 256 for single dimension convolution layer (**Conv1D**) filter size, 4 and 8 for **MaxPooling window size**, 256 and 512 for **LSTM units**, 64 and 128 for **Dense units** and 1 and 4 for **training epochs**. We infer the influence of each hyper-parameter by measuring the difference in training/prediction time and recall at fixed FPR of 10^{-3} between a pair of configuration sets for that particular hyper-parameter. For example, for evaluating the options for Conv1D, one pair is formed by the tuples (128, 4, 256, 64, 1) and (256, 4, 256, 64, 1), and so on. Based on sixteen such pairs of tuples per hyper-parameter, we compute the average differences in recall, training and prediction times, denoted as Δr, Δt, and Δp, respectively. Table 9 summarizes the performance differences.

Computational overhead and accuracy. As seen in Table 9, the change of Epoch results in the largest and only significant increase in recall (at fixed FPR of 10^{-3}), which is 0.1592 on average. Indeed, of the 32 different configuration tuples, the biggest jump in recall, of 0.6912, is observed between configurations which have Conv1D:128, MaxPooling:4, LSTM:512, Dense:64 (in common), but different values for **Epoch**. However, the gain in recall comes at the cost of increased computational time. Since Epoch is the number of times the entire dataset is iterated over, the training time increases with the number of iterations; however, this also leads to higher probability in minimizing the error between the input and the output of the neural network.

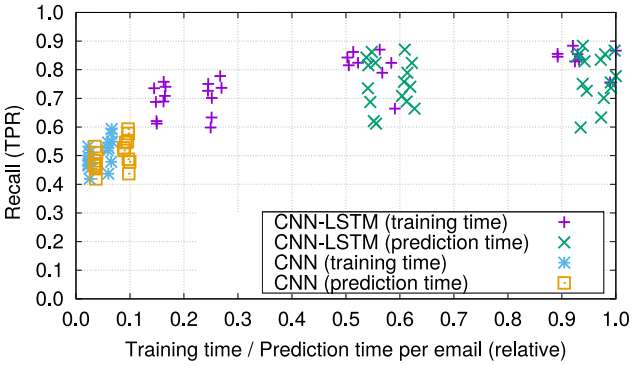


Figure 10: URL module: Recall at FPR of 10^{-3} v/s relative training and prediction time per email, for various neural network model configurations.

Another hyper-parameter that causes a positive impact is the **MaxPooling window size**. A larger pooling window significantly reduces the time for training and prediction, by pooling a larger feature space in a single operation. On the other hand, increasing Conv1D filter size increases the convolution operation complexity. Similarly increasing the number of LSTM units leads to more computational time during both forward pass and back-propagation.

Fig. 10 plots the recall at fixed FPR of 10^{-3} as well as the training and prediction time for the CNN-LSTM models corresponding to the 32 configurations in Table 9, and 16 CNN models with the same hyper-parameter values (excluding those for LSTM). Thus, each pair of points (one for training and another for prediction) on the plot corresponds to a unique set of configurations of the model hyper-parameters. Fig. 10 clearly shows the cost-efficient configurations, i.e., the points with a higher recall without notable increase in training/prediction time. For example, the model configuration (128, 8, 512, 64, 4) achieves 0.86 recall at 10^{-3} FPR, but taking nearly half the training and prediction time (therefore plotted on the top-middle of the figure) of a costly configuration having a similar recall, e.g., (128, 4, 256, 128, 4).

6.5. Meta-classification alternatives

The meta-classifier in D-Fence could also be substituted with simpler functions, such as one taking the mean or maximum of the predictions of the individual modules. Note that the individual predictions are probability values between 0.0 and 1.0. Taking a naive mean of these values or the highest probability as the decision probability are likely less accurate than an ML-trained meta-classifier. In our experiments, at 10^{-3} FPR, using mean and max results in 0.9653 and 0.8628 of recall, respectively, whereas the RandomForest model achieves a recall of 0.9932.

6.6. System-wise overhead-accuracy trade-off

Next, we analyze the system-wise performance of D-Fence with the combinations of different configurations

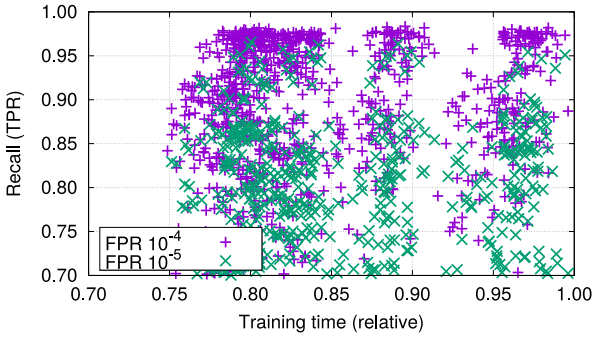


Figure 11: D-Fence: Recall (TPR) at fixed FPRs of 10^{-4} and 10^{-5}, and relative training time for the different configurations of structure module and URL module. Configuration of text module is fixed at 100 input words.

of the individual analysis modules. That is, the different settings are due to 1024 feature sets in structure module (Sec. 6.2), five input word lengths in text module (Sec. 6.3), and 32 hyper-parameter configurations for CNN-LSTM and 16 configurations for CNN in URL module (Sec. 6.4). From each such setting, we obtain the prediction probabilities on $\mathcal{D}_{ts}$ from the analysis modules and test using the meta-classier model trained on $\mathcal{D}_{ts}$. For measuring the training time and prediction time, we summarize the absolute time taken for the end-to-end process of each setting (e.g., the total training time is the sum of the training times of each module in their respective configurations).

The slowest module of D-Fence is the text module, caused by the overhead due to the extraction of BERT embeddings. Specifically, comparing the time for training and prediction with 500 words (which is the slowest configuration for text module), reduction of input word count by steps of 100 decreases the training and prediction time to 78% (400 words), 55% (300 words), 36% (200 words) and 22% (100 words), all without considerable loss of recall at a fixed FPR of 10^{-3}. Note that, the reduction in computational time is due to reduced extraction time for BERT embeddings. Based on this observation, we narrow down the settings by fixing the number of input words in text module at 100 words and exploring different configurations for structure module and URL module. This word reduction reduces the training time from 20 hours to 5 hours on EES 2020 in our experimental environment.

Fig. 11 plots recall at fixed FPRs of 10^{-4} and 10^{-5} and the relative training time, for different settings of D-Fence. We observe that we can achieve a reduction of training time by 20%, while not sacrificing the detection capability. As examples of selected settings, we see tens of combinations which satisfy 20%+ training time reduction, while achieving 0.975%+ recall at 10^{-4} FPR and 0.95%+ recall at 10^{-5} FPR. In absolute time, it further decreases the overall training from 5 hours to 4 hours for $\mathcal{D}_{tr}$ of EES 2020. One corresponding setting has XGBoost model with a feature set excluding `CSS`, `JS`, `Link`, and `Section boundary` feature categories for structure module, XGBoost model with 100 input words for text module, and (Conv1D:128, MaxPooling:8, LSTM:256, Dense:128, Epoch:1) CNN-LSTM configuration for URL module. This cost-efficient configuration also reduces the prediction time by 5% (see Fig. 17 in Appendix A.4). Interestingly, this high recall is achieved with the module configurations that do not form the best-recall configuration in each individual module. For example, the configuration for structure module has 0.95 of recall at 10^{-3} FPR with 20% reduced training/prediction time, which is 0.03 less than the best recall configuration in that particular module. Similarly, in URL module, models with 1 training epoch reduce the training time but lead to less accuracy. This implies that a synergistic combination provides even higher system-wise detection capability than that achievable by the combination of the best configurations of individual modules.

7. Related Works

Early works on phishing email detection relied on defining rules that differentiate phishing characteristics from benign ones [12], [51]. The rule-based detection approaches are usually efficient and explainable, and therefore have been applied in commercial solutions despite its most critical drawback, i.e., high dependency on experts to continuously define and refine rules according to changing attack landscape. Previous studies also investigated reputations of sender characteristics; e.g., RepuScore [67] utilizes sender's reputation to classify spam and phishing emails. Although reputation-based approaches are effective in distinguishing fake senders impersonating genuine users, their performance depends heavily on the external reputation sources, i.e., DNS servers, WHOIS database, third-party IP/domain blacklist, etc. And reputation scores generally take considerable time to be built, as they are based on observations made over a period of time.

The approaches employing static rules and blacklists, etc. lag in catching up with the fast-evolving phishing attack ecosystem [1], [54]. This has led to the development of ML-based classification models to automatically learn the discriminating characteristics of phishing emails. Such models use features that capture lexical aspects of email body (e.g., length of text, the number of words, spaces and dots) [22] and embedded URLs [48], existence of sensitive words [75], and semantics of text using WordNet [69]. There are also research works that adopt a wide range of stylometric and contextual features (along with reputations) as a large feature pool and build an optimized feature set and model through feature selection [32], [76].

The work by Verma et al. [70] shares our threat model, aiming to have the same coverage. Although they explore NLP techniques to develop the detect phishing emails, their solution is still based on scores and rules (e.g., a mail having no text is deemed as phishing) that have limitations as above. The more recent studies applying NLP techniques and/or deep learning (DL) techniques attempt to differentiate phishing emails based on the text contents [25], [30]. The shortcoming of these solutions is the limited attack surface they cover, since phishing emails could avoid text or even have very short texts. Replacing a text and a HTML page into an image to bypass a detection system is also a well-known evasion technique.

Similarly, DL models have also been applied to classify malicious URLs without accessing external reputation scores, and more importantly, without accessing the web-contents corresponding to the URLs [10], [36], [43]. Such

models are shown to have good detection capability, while also being very efficient (only a string of characters need to be processed); however, classification based purely on URL strings lacks robustness against evasive URLs [4].

Recently, Ho et al. studied the characteristics of lateral phishing emails (i.e., phishing emails send from legitimate but compromised enterprise accounts) based on a large-scale dataset coming from several organizations [33], [34]. However, the lateral phishing solution targets only emails between employee accounts; therefore, the detection features are specific to that context, and not applicable to general phishing email scenarios. For example, [33] uses per-account historical emails to compute features such as Jaccard's similarity (capturing differences between specific characteristics of a given email and historical emails from the sender).

D-Fence is motivated by the shortcomings of existing phishing email detection methods, i.e., limited coverage of attack surface, inherent limitations of standalone models, dependence on external reputation scores, etc. Therefore, D-Fence is designed as a multi-modular phishing detection system with flexible configurations, that processes and classifies emails independently, requiring neither a reputation scoring system nor learning from user-specific historical emails.

8. Discussion

Reproducibility. One can reproduce D-Fence by building individual modules with the hyper-parameters we disclose in Sec. 5.4. All the ML/DL algorithms (e.g., RandomForest, Convolution layer, and LSTM layer) used in each classification model and the pre-trained transformer, i.e., BERT, are available from public implementations, specifically, Python's scikit-learn library [55]. For structure module, we provide the full list of features in Table 10 in Appendix A.5.

Adversarial attacks and countermeasures. Adversarial attack can be performed using two attack vectors: adversarial training and adaptive evasion. An attacker may carefully craft benign emails that are similar to her future phishing emails, or send a large amount of biased phishing emails to influence the bias in the training set. Although the architecture of D-Fence does not cover the evaluation and selection method for the training set, the threat from adversarial attacks can be reduced by off-line sanitization process and preserving diversity of training set and models systematically. In particular, temporal diversity is a key advantage to combat adversarial attacks since an attacker cannot poison the past sample sets.

On the other hand, an attacker who is aware of D-Fence may generate adaptive phishing emails. Adversarial texts for deceiving a text analysis model and spear-phishing emails that are hardly distinguishable from the benign emails in their structure can be threats to D-Fence. Specifically, each module can be threatened by following adaptive attacks; *Structure*: spear-phishing emails sent through a normal email application individually; *Text*: substitution of words and characters, and creation of intentional typos [46] to deceive the model; *URL*: shortened URLs and hosting malicious contents on the URLs of legitimate services like cloud services. Since individual spear-phishing emails can be perfectly legitimate in terms of the structure, the final decision naturally relies on the text and URL module. While we do not have comprehensive results on tackling adversarial attacks, D-Fence did detect sets of adaptive phishing emails. These include emails that not only change links, but also change texts gradually. We refer to Appendix C for two such examples. Yet, we would like to explore some countermeasures in future. i) Spelling check to standardize the manipulated text [46], at the same time defining features that capture use of typos and language errors. ii) Training a robust text model with adversarial training samples against known attack methods [71]. Meanwhile, malicious content behind a legitimate-looking URL can be addressed by content-analysis solutions, e.g., [60], [73]. Since these solutions require additional computation and time to access and analyze the actual contents, a future D-Fence may have a layered decision architecture for selective investigation.

Text length limitation. The *bert-base* transformer in our text module limits the analysis to 512 tokens; although the use of 512 tokens covers 91% of the EES 2020 dataset. Increasing the number of tokens would require another model. In future, we plan to explore the recent advancements (e.g., [21], [45]) for both increased tokens and faster embedding.

Multi-language support. The *bert-base* transformer in D-Fence supports only English language. However, additional language support can easily be achieved by using a multi-language model, e.g., *bert-base-multilingual* [26] or a language agnostic transformer, e.g., Language-Agnostic Sentence Representations (LASER) [62].

Limitations of evaluations. Though we evaluate D-Fence with a large-scale real-world dataset and public dataset as well, there are a few limitations. The experimental results on EES 2020 dataset are not directly comparable with future studies since EES 2020 dataset is private. Regarding the cost-efficient and best-performing configuration, we show that the modularized system can flexibly reduce its training/testing cost; however, these configurations may not be generally applicable to other environments and datasets. In general, as is well-known, learning models have to be tuned for specific datasets and for target cost (modeling/prediction time).

9. Conclusion

We developed and presented D-Fence, a multi-modular, comprehensive, and flexible phishing email detection system. With the three independent analysis modules — structure module, text module, and URL module — D-Fence is able to cover a larger attack surface than existing solutions. Our evaluations on real-world enterprise email dataset show that D-Fence achieves high detection capability while providing flexibility in reducing computational overhead.

Acknowledgment

This research is supported by the National Research Foundation, Prime Minister's Office, Singapore under its Corporate Laboratory@University Scheme, National University of Singapore, and Singapore Telecommunications Ltd.

References

[1] Akamai. Phishing is no longer just email: It's social. https://www.akamai.com/us/en/multimedia/documents/white-paper/phishing-is-no-longer-just-email-its-social-white-paper.pdf, Aug. 2019 [Accessed: Oct 2020].

[2] Andronicus A Akinyelu and Aderemi O Adewumi. Classification of phishing email using random forest machine learning technique. *Journal of Applied Mathematics*, 2014.

[3] Samar Al-Saqqa and Arafat Awajan. The use of Word2Vec model in sentiment analysis: A survey. In *Proc. of Int'l. Conf. on Artificial Intelligence, Robotics and Control*, pages 39–43, 2019.

[4] Ahmed AlEroud and George Karabatis. Bypassing Detection of URL-based Phishing Attacks Using Generative Adversarial Deep Neural Networks. In *Proc. of Int'l Workshop on Security and Privacy Analytics (IWSPA)*, pages 53–60, 2020.

[5] Ammar Almomani, Tat-Chee Wan, Altyeb Altaher, Ahmad Manasrah, Eman ALmomani, Mohammed Anbar, Esraa ALomari, and Sureswaran Ramadass. Evolving fuzzy neural network for phishing emails detection. *Journal of Computer Science*, 8(7):1099, 2012.

[6] Kurt Andersen, Brandon Long, Seth Blank, and Murray Kucherawy. The authenticated received chain (ARC) protocol. RFC 8617, RFC Editor, July 2019.

[7] Ben Athiwaratkun and Jack W. Stokes. Malware classification with LSTM and GRU language models and a character-level CNN. In *Proc. of IEEE Int'l. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.

[8] Avanan. Global phish report 2019. https://www.avanan.com/global-phish-report, Apr. 2019 [Accessed: Oct 2020].

[9] Dave Baggett. Text direction deception! Technical report, Jul 2019.

[10] Alejandro Correa Bahnsen, Eduardo Contreras Bohorquez, Sergio Villegas, Javier Vargas, and Fabio A González. Classifying phishing URLs using recurrent neural networks. In *Proc. of APWG Symposium on Electronic Crime Research (eCrime)*, pages 1–8. IEEE, 2017.

[11] Ram Basnet, Srinivas Mukkamala, and Andrew H Sung. Detection of phishing attacks: A machine learning approach. In *Soft computing applications in industry*, pages 373–383. Springer, 2008.

[12] Ram B Basnet, Andrew H Sung, and Quingzhong Liu. Rule-based phishing attack detection. In *Proc. of Int'l. Conf. on Security and Management (SAM)*, 2011.

[13] Aaron Blum, Brad Wardman, Thamar Solorio, and Gary Warner. Lexical feature based phishing URL detection using online learning. In *Proc. of ACM Workshop on Artificial Intelligence and Security (AISEC)*, pages 54–60, 2010.

[14] Anna L Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications surveys & tutorials*, 18(2):1153–1176, 2015.

[15] Girish Chandrashekar and Ferat Sahin. A survey on feature selection methods. *Computers & Electrical Engineering*, 40(1):16–28, 2014.

[16] Asaf Cidon, Lior Gavish, Itay Bleier, Nadia Korshun, Marco Schweighauser, and Alexey Tsitkin. High precision detection of business email compromise. In *Proc. of USENIX Security Symposium*, pages 1291–1307, 2019.

[17] William W. Cohen. Enron email dataset. https://www.cs.cmu.edu/~./enron/, 2015 [Accessed:Aug 2020].

[18] Symantec Corporation. Internet Security Threat Report. https://docs.broadcom.com/doc/istr-24-2019-en, Feb. 2019 [Accessed: Oct 2020].

[19] Jesse Davis and Mark Goadrich. The relationship between Precision-Recall and ROC curves. In *Proc. of ACM Int'l. Conf. on Machine Learning (ICML)*, pages 233–240, 2006.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. of Conf. on the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1–16, 2019.

[21] Ming Ding, Chang Zhou, Hongxia Yang, and Jie Tang. CogLTX: Applying BERT to long texts. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] Sevtap Duman, Kubra Kalkan-Cakmakci, Manuel Egele, William Robertson, and Engin Kirda. Emailprofiler: Spearphishing filtering with header and stylometric features of emails. In *Proc. of Annual Computer Software and Applications Conf. (COMPSAC)*, volume 1, pages 408–416. IEEE, 2016.

[23] Gal Egozi and Rakesh Verma. Phishing email detection using robust NLP techniques. In *Proc. Int'l Conf. on Data Mining Workshops (ICDMW)*, pages 7–12. IEEE, 2018.

[24] Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings. *arXiv preprint arXiv:1909.00512*, 2019.

[25] Yong Fang, Cheng Zhang, Cheng Huang, Liang Liu, and Yue Yang. Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism. *IEEE Access*, 7:56329–56340, 2019.

[26] Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. Language-agnostic bert sentence embedding. *arXiv preprint arXiv:2007.01852*, 2020.

[27] Ian Fette, Norman Sadeh, and Anthony Tomasic. Learning to detect phishing emails. In *Proc. of Int'l. Conf. on World Wide Web (WWW)*, pages 649–656, 2007.

[28] The Apache Software Foundation. *SpamAssassin Old Public Corpus Index*, 2002 [Accessed: Aug 2020].

[29] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36:193–202, 1980.

[30] Lukáš Halgaš, Ioannis Agrafiotis, and Jason RC Nurse. Catching the phish: Detecting phishing attacks using recurrent neural networks RNNs. In *Proc. of Int'l. Workshop on Information Security Applications (WISA)*, pages 219–233. Springer, 2019.

[31] Isredza Rahmi A. Hamid and Jemal Abawajy. Hybrid feature selection for phishing email detection. In *Proc. of Int'l. Conf. on Algorithms and Architectures for Parallel Processing*, pages 266–275. Springer-Verlag, 2011.

[32] Isredza Rahmi A Hamid, Jemal Abawajy, and TH Kim. Using feature selection and classification scheme for automating phishing email detection. *Studies in informatics and control*, 22(1):61–70, 2013.

[33] Grant Ho, Asaf Cidon, Lior Gavish, Marco Schweighauser, Vern Paxson, Stefan Savage, Geoffrey M Voelker, and David Wagner. Detecting and characterizing lateral phishing at scale. In *Proc. of USENIX Security Symposium*, pages 1273–1290, 2019.

[34] Grant Ho, Aashish Sharma, Mobin Javed, Vern Paxson, and David Wagner. Detecting credential spearphishing in enterprise settings. In *Proc. of USENIX Security Symposium*, pages 469–485, 2017.

[35] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, Nov 1997.

[36] Yongjie Huang, Qiping Yang, Jinghui Qin, and Wushao Wen. Phishing URL detection via CNN and attention-based hierarchical RNN. In *Proc. IEEE TrustCom*, pages 112–119, 2019.

[37] Rafiqul Islam and Jemal Abawajy. A multi-tier phishing detection and filtering approach. *Journal of Network and Computer Applications*, 36(1):324–335, 2013.

[38] Ming-Wei Chang Jacob Devlin and Kenton Lee. BERT GitHub repository. https://github.com/google-research/bert, 2020.

[39] Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. Is BERT Really Robust? A Strong Baseline for Natural Language Attack on Text Classification and Entailment. *arXiv*, pages arXiv–1907, 2019.

[40] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. Bag of tricks for efficient text classification. *arXiv preprint arXiv:1607.01759*, 2016.

[41] Siwei Lai, Liheng Xu, Kang Liu, and Jun Zhao. Recurrent convolutional neural networks for text classification. In *Proc. AAAI*, 2015.

[42] Anh Le, Athina Markopoulou, and Michalis Faloutsos. Phishdef: URL names say it all. In *Proc. IEEE INFOCOMM*, 2011.

[43] Hung Le, Quang Pham, Doyen Sahoo, and Steven CH Hoi. URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*, 2018.

[44] Jehyun Lee, Pingxiao Ye, Ruofan Liu, Dinil Mon Divakaran, and Chan Mun Choon. Building robust phishing detection system: an empirical analysis. In *Proc. of NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, February 2020.

[45] Younghoo Lee, Joshua Saxe, and Richard Harang. CATBERT: Context-aware tiny BERT for detecting social engineering emails. *arXiv preprint arXiv:2010.03484*, 2020.

[46] Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. Textbugger: Generating adversarial text against real-world applications. *arXiv preprint arXiv:1812.05271*, 2018.

[47] Joseph Lilleberg, Yun Zhu, and Yanqing Zhang. Support Vector Machines and Word2Vec for text classification with semantic features. In *Proc. IEEE Int'l. Conf. on Cognitive Informatics & Cognitive Computing (ICCI* CC)*, pages 136–140, 2015.

[48] Samuel Marchal, Jérôme François, Radu State, and Thomas Engel. Phishstorm: Detecting phishing with streaming analytics. *IEEE Trans. Netw. Service Manag.*, 11(4):458–471, 2014.

[49] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proc. of Int'l. Conf. on Learning Representations (ICLR) Workshop*, 2013.

[50] Jason Milletary. Technical trends in phishing attacks. 2005.

[51] Mahmood Moghimi and Ali Yazdian Varjani. New rule-based phishing detection method. *Expert systems with applications*, 53:231–242, 2016.

[52] Jose Nazario. *Phishcorpus*, 2004 [Accessed: Aug 2020].

[53] Nathaniel S. Borenstein Ned Freed. Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies. RFC 2045, RFC Editor, 11 1996.

[54] Adam Oest, Yeganeh Safaei, Penghui Zhang, Brad Wardman, Kevin Tyers, Yan Shoshitaishvili, and Adam Doupé. PhishTime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists. In *Proc. of USENIX Security Symposium*, pages 379–396, 2020.

[55] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[56] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018.

[57] Anti phishing working group. Phishing activity trends reports-2nd quarter 2020. https://docs.apwg.org/reports/apwg_trends_report_q2_2020.pdf, Aug. 2020 [Accessed: Oct 2020].

[58] Vinayakumar Ra, Barathi Ganesh HBa, Anand Kumar Ma, Soman KPa, Prabaharan Poornachandran, and A Verma. Deepantiphishnet: Applying deep neural networks for phishing email detection. In *Proc. of ACM Int'l. Workshop on Security and Privacy Analytics (IWSPA)*, pages 1–11. Tempe, AZ, USA, 2018.

[59] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[60] Routhu Srinivasa Rao and Alwyn Roshan Pais. Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Computing and Applications*, 31(8):3851–3873, 2019.

[61] Tomas Mikolov Razvan Pascanu and Yoshua Bengio. Understanding the exploding gradient problem. *Computing Research Repository (CoRR)*, abs/1211.5063, 2012.

[62] Facebook Research. Language-agnostic sentence representations. https://github.com/facebookresearch/LASER, Feb 2019 [Accessed: Feb 2021].

[63] Peter W. Resnick. Common internet message headers. RFC 2076, RFC Editor, 2 1997.

[64] Peter W. Resnick. Internet message format. RFC 2822, RFC Editor, 4 2001.

[65] Angelo PE Rosiello, Engin Kirda, Fabrizio Ferrandi, et al. A layout-similarity-based approach for detecting phishing pages. In *Proc. of Int'l. Conf. on Security and Privacy in Communications Networks and the Workshops (SecureComm)*, pages 454–463. IEEE, 2007.

[66] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. *Learning Internal Representations by Error Propagation*, pages 318–362. MIT Press, Cambridge, Jan 1986.

[67] Gautam Singaraju and Brent ByungHoon Kang. RepuScore: Collaborative reputation management framework for email infrastructure. In *Proc. of USENIX Large Installation System Administration Conf. (LISA)*, volume 7, pages 243–251, 2007.

[68] Lisa Vaas. Phishing is still the most commonly used attack on organizations, survey says. https://nakedsecurity.sophos.com/2018/10/23/phishing-is-still-the-most-commonly-used-attack-on-organizations-survey-says/, Oct. 2018 [Accessed: Oct 2020].

[69] Rakesh Verma and Nabil Hossain. Semantic feature selection for text with application to phishing email detection. In *Proc. of Int'l. Conf. on Information Security and Cryptology (ICISC)*, pages 455–468. Springer, 2013.

[70] Rakesh Verma, Narasimha Shashidhar, and Nabil Hossain. Detecting phishing emails the natural language way. In *Proc. of European Symposium on Research in Computer Security (ESORICS)*, pages 824–841. Springer, 2012.

[71] Jason Wei and Kai Zou. Eda: Easy data augmentation techniques for boosting performance on text classification tasks. *arXiv preprint arXiv:1901.11196*, 2019.

[72] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[73] Guang Xiang, Jason Hong, Carolyn P Rose, and Lorrie Cranor. CANTINA+ a feature-rich machine learning framework for detecting phishing web sites. *ACM Transactions on Information and System Security (TISSEC)*, 14(2):1–28, 2011.

[74] Peng Yang, Guangzhen Zhao, and Peng Zeng. Phishing Website Detection Based on Multidimensional Features Driven by Deep Learning. *IEEE Access*, 7:15196–15209, 2019.

[75] Adwan Yasin and Abdelmunem Abuhasan. An intelligent classification model for phishing email detection. *arXiv preprint arXiv:1608.02196*, 2016.

[76] Masoumeh Zareapoor and KR Seeja. Feature extraction or feature selection for text classification: A case study on phishing email detection. *Int'l. Journal of Information Engineering and Electronic Business*, 7(2):60–65, 2015.

[77] Ziyun Zhu and Tudor Dumitraş. FeatureSmith: Automatically engineering features for malware detection by mining the security literature. In *Proc. ACM CCS*, pages 767–778, 2016.

Appendix A.
Figures and tables

A.1. Email components: an example

Fig. 12 illustrates an example of email with multiple email components.

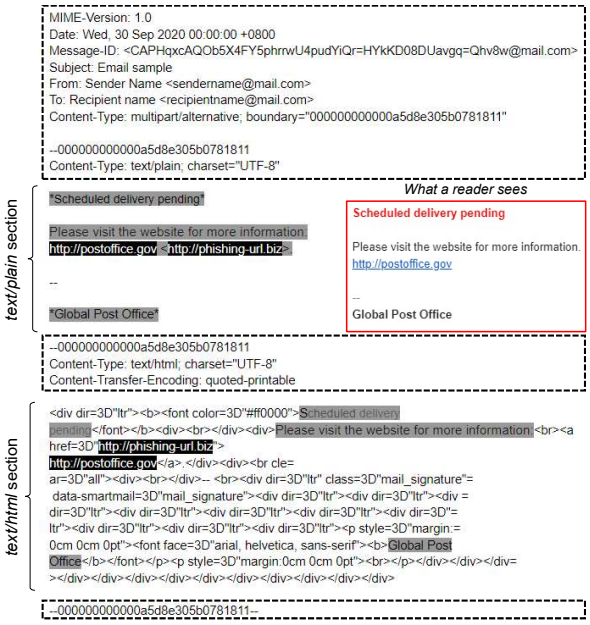


Figure 12: Example of an email with email headers, plain text, URLs and HTML components. We mark the email headers with dotted boxes. Gray-shaded texts are readable texts in text/plain and text/html sections. Black-shaded texts are URLs.

A.2. CNN-LSTM architecture

Fig. 13 illustrates the CNN-LSTM architecture used for URL module in Sec. 4.3.

A.3. Performance of different modules based on the incorporated models

Fig. 14, 15 and 16 plot the recall for different values of FPRs, for different modeling techniques in, structure module, text module, and URL module, respectively. In the first two figures, and in particular in Fig. 15, Naive Bayes classifier has lesser points due to the less number of prediction probabilities it computes.

A.4. Evaluation of D-Fence at reduced computational overhead

Fig. 17 plots the recall (at fixed FPRs) and prediction time, for different module configurations of D-Fence.

A.5. Feature list for structure module

Table 10 lists the features used by the structure module in D-Fence. In addition to trivial features, some of the

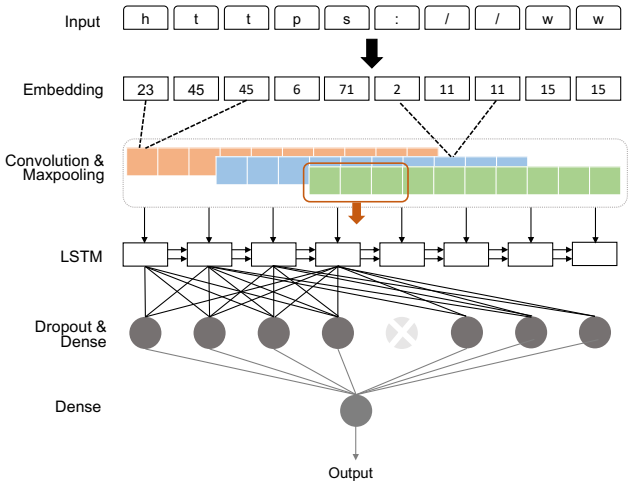


Figure 13: Architecture of CNN-LSTM URL classification model

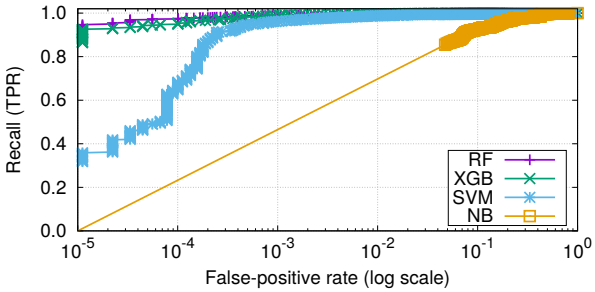


Figure 14: Performance of models in structure module

features are inspired from previous studies on phishing detection, e.g., [2], [5], [9], [11], [22], [27], [37], [65], [73]. We highlight the novel features that we introduce based on our domain-knowledge and empirical observations with a check mark (✓).

Appendix B.
False prediction analysis

First, we look into the false positive/negative cases due to one analysis module while the other two modules and the meta-classifier make a correct prediction. The phishing emails in these cases could evade detection if we use one module solely or when the other data source (i.e., text and URL) are not available.

B.1. False predictions by structure module

False negatives. We analyze the common characteristic of phishing emails which evade structure module, in terms of email header and HTML features. We observe that the false negatives commonly have more than three `Received` headers that indicate multiple intermediate mail servers between the sender and the recipient. They also commonly use an identical character set in `text/plain` and `text/html` sections, when both sections are present. These are similar characteristics of benign emails. The last noticeable tendency in these false negatives is regarding the feature `unique domain ratio`. For example, two unique domain names in twenty

TABLE 10: Feature list of structural module

Category	Feature	Note
Section statistics (mandatory)	Number of `text/plain` sections	
	Number of `text/html` sections	
	Number of `image` sections	
	Number of `application` sections	
	Ratio of `text/plain` to any `text` sections	
	Length of texts in `text/html` section	[22]
Header statistics	Number of standard headers	
	Number of `In-Reply-To` entities	
	Number of `Received` entities	✓
	Existence of `User-Agent`	✓
	Existence of `X-mailer`	[22]
MIME version	MIME version of the first section	[22]
	Case variance in `MIME-Version` header tag	✓
Message-ID (e.g., {ID@domain})	Length of `Message-ID`	✓
	ASCII number of `Message-ID` boundary character (if any)	✓
	Existence of the domain of `Message-ID` (if any) in ID part	✓
	Is ID part hexadecimal (except the special characters)	✓
	Is ID part decimal (except the special characters)	✓
	Existence of dots in ID part	✓
	Existence of special characters in ID part	✓
	Is domain part hexadecimal (except the special characters)	✓
	Is domain part decimal (except the special characters)	✓
	Existence of dots in domain part	✓
	Existence of special characters other than dots in domain part	✓
Mail address and domain	Number of `From` header tags in the mail thread	[37]
	Number of `To` header tags in the mail thread	[37]
	Number of unique `To` addresses in the mail thread	
	Number of unique `Reply-To` addresses in the mail thread	
	Number of unique domains in all email addresses	
	Ratio of unique `To` domains to the unique domains in all email addresses	
	Number of `Cc` addresses	[22]
	Number of `Bcc` addresses	[22]
	Number of `Sender` addresses	[22]
	Is `Return-Path` addresses identical to `Reply-To` addresses	[5]
	Number of `To` domains identical to `From` domain	[5]
	Number of `Cc` domains identical to `From` domain	[5]
	Number of `Bcc` domains identical to `From` domain	[5]
	Is `From` domain same to `Reply-To` domain	[5]
	Is `From` domain same to `Return-Path` domain	[5]
	Is `From` entity bracketed	✓
	Is `To` entity bracketed	✓
Section boundary	Length of the first `boundary` ID	✓
	Does the first `boundary` ID start with an equal symbol (=)	✓
	Existence of equal symbols (=) in the middle of the first `boundary` ID	✓
	Existence of underscores in the first `boundary` ID	✓
	Existence of dots in the first `boundary` ID	✓
	Existence of the other special characters in the first `boundary` ID	✓
	Is the first `boundary` ID hexadecimal (except the special characters)	✓
	Is the first `boundary` ID decimal (except the special characters)	✓
Character set	Index of `charset` in the first section	✓
	Number of unique `charset` in all the sections	✓
Cascading Style Sheets (CSS)	Length of `<style>` bodies and inline `style` bodies	
	Existence of `direction: rtl` style for backward rendering	[9]
JavaScript (JS)	Existence of inline `<script>`	[2], [11], [27]
Document Object Model (DOM)	Depth of DOM object tree	[65], [73]
	Number of DOM leaf nodes	[65], [73]
	Number of unique DOM leaf node types	[65], [73]
	Mean of DOM leaf node depths	[65], [73]
	Standard deviation of DOM leaf node depths	[65], [73]
Links	Number of email addresses in `text/html` section	
	Number of `<a href>` and `<a data-saferedirecturl>` tags	[11], [27]
	Number of URLs in `text/html` section	[2]
	Ratio of unique domains to the domains of all URLs in any tags	[2], [11], [27]

URLs results in this feature having a value of 0.1 (two unique domains out of twenty domains), whereas the feature value is 1.0 when only two URLs exist in total. Therefore, if a phishing sample has many URLs, this ratio

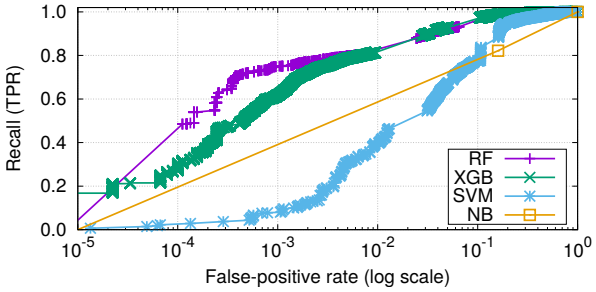


Figure 15: Performance of models in text module

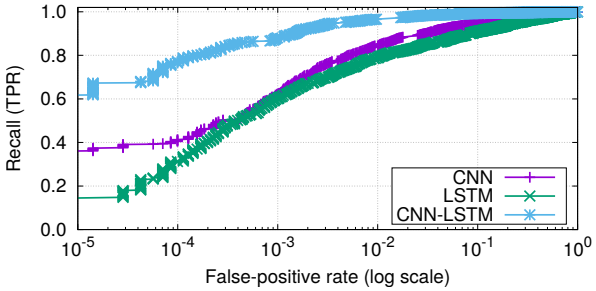


Figure 16: Performance of models in URL module

gets closer to the values of benign samples. While this may suggest designing better features, it is important to note that such features are also the ones that contribute towards detection.

False positives. Coming to the false positive cases, given that the structure module has only a few of them, it is not surprising that most of them have short or no HTML sections. Therefore, (in such cases) module prediction is dependent on the email headers. The email headers in false positive samples commonly share the following rare characteristics of having i) a small number of standard headers (described in Sec. 3.1), ii) three or less `Received` headers, iii) different domain names in `From` and `Reply-To` email addresses, iv) undisclosed-recipients on `To` header, and v) high unique domain ratio close to 1.0 (e.g., five URLs under five different domain names) across the URLs in headers and HTML sections.

We highlight that the above false predication are correctly classified by the other two modules and our over overall detection system D-Fence (meta-classifier). This indeed showcases the benefit of our comprehensive D-Fence system over an independent individual module.

B.2. False predictions by text module

Both false positives and false negatives by text module are mainly due ot short sentences. When a sentence is short, and majority of the words have neutral sentiment (e.g., name of a company) the prediction would be limited and driven by a few sensitive words.

False negatives. In our observation, we find two common characteristics in the false negatives: i) implicit instruction, and ii) email structure that evades the scope of our text module. Texts with implicit instruction have few verbs asking for any further action. A phishing URL, if present, is placed independent from the main text and is not explicitly pointed by the use of verbs (e.g., check, review, see, and open). However, implicitly, the text and subject

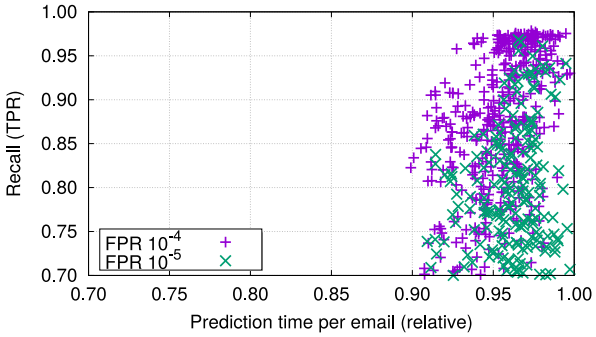


Figure 17: D-Fence: Recall (TPR) in the given 10^{-4} and 10^{-5} FPR, and relative prediction time per email by combination of the structure and URL module configurations. Configuration for text module is fixed as 100 input words.

of the email leads a reader to click the URL or reply to the email. Structural evasion cases place their key messages in HTML objects. As our text analysis does not consider the parameters of HTML tags as a readable text, those emails evade our text module. For example, `CLICK HERE` is considered as a text in the following example,

```
<a href="http://phishing.url">CLICK
HERE</a>
```

but not in the following example, where instead `LINK` is extracted as the text.

```
<a title="CLICK HERE"
href="http://phishing.url">LINK</a>
```

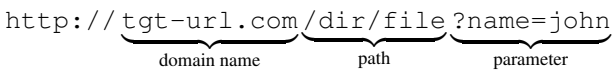
In marginal cases, we also find very long text in the false negatives. With long text, only a couple of sentences are related to the intention of phishing, and all the others are verbose sentences disguising as a formal business email. Naturally, the phishing intention is possibly blurred by the other neutral sentences.

False positives. The two common tendencies in false positive cases are: i) words which do not make a sentence, and ii) suspicious writing even for a human reader. False positive emails commonly have many individual words which are not part of a sentence; a table in text format is an example. Since text module considers a text as an one-dimension sequence of words, the words on columns and rows of a table form a word sequence with strange composition. Moreover, most of the text models, not limited to BERT, ignore the numbers between words. Following is a real-world example of a sequence of words we get after preprocessing: `[cost,insurance,discount,extras,total, details,sales,book,refund,deposit]`. Clearly, this does not appear like a sentence, therefore BERT cannot generate useful embeddings for this.

The second and perhaps the most challenging characteristic of false positive is the very phishing-like benign emails without any related context. Some false positive samples are full of urgent and repeated instructions with only a few neutral words. The sentiment of these samples is very close to typical phishing emails if we do not have background knowledge of the sender and the recipient.

B.3. False predictions by URL module

False predictions by the URL module is totally dependent on the characters on a URL string, since the DL model in this module uses only the characters as its feature space. We observe that common characteristics of false predictions are related to strings in the path and the parameter which are sub-strings of a URL.



The common characteristic of false negatives is the presence of readable and existing English words in the domain name and path of the URL. They commonly do not have any parameters following the domain name and path, making the URLs appear like benign.

In contrast, we see random alphanumeric strings on the paths of the false positive URLs. Random alphanumeric strings (e.g., `8Xj13EaP`) are rarely observed in the path of benign URLs but often in the phishing URLs, such as shortened URLs. Note, however, that the random hexadecimal or decimal numbers (e.g., `A03FE389`) are commonly used by benign and phishing URLs for indicating hashes and hexa-encoded strings.

B.4. False predictions by meta-classifier

Though the analysis modules supplement each other, the meta-classifier still has a small number of mispredictions. If majority of the analysis modules mispredict an email sample, unsurprisingly it leads to the misprediction at the meta-classifier. However, since the meta-classifier learns the prediction probabilities of the analysis modules as its modelling features, a module which provides more accurate and clearly distinct values between two classes gets more weight for the final prediction. The meta-classifier learns this tendency from its training set. Therefore, for low-confident predictions, the structure module has more weight than other modules. This aligns well with our analysis of wrong predictions by the meta-classifier that, false negatives are largely affected by the structure module module, whereas false positives do not have any strong bias towards any module.

Appendix C. Adversarial examples

Example 1: Legitimate structure with text variants. We observe evasive emails which have normal email headers and very simple HTML structure. At the same time, those emails have various forms of sentences that hinder the text analysis. The three emails in Fig. 18 are examples of evasive emails out of hundreds of variants conveying the same key message. Despite their legitimate structure, these phishing emails were detected by D-Fence with high score from text module and URL module.

Example 2: Text variants without URL links. The three emails in Fig. 19 are examples from a phishing email set that has wide variance in the texts. Tens of emails in this set coax the victims to reply with their personal information; however these mails do not have (phishing) URL links. The evasion techniques employed range from word substitutions to rewriting in formal expressions. For instance, the first two samples differ in only a few sentence organizations, words, and fund amounts. However, the third sample attempts to use much longer sentence with many redundant and neutral words that likely hinder the text analysis. These emails were detected with high scores from structure module and text module.

15 of your incoming messages has been suspended because your email box account needs to be verify now.
Do**<link>** Verify**<link>** now to release your messages

Microsoft Verification Team

Alot of your incoming messages has been suspended now because your email box account has not been verify for this year 2020.
Please do verify**<link><link><link>** now so that all your suspended incoming messages can be released to you now.
Thanks for your understanding

Microsoft Verification Team

Your email box account needs to be upgraded **<link>** now to our latest version of Microsoft Outlook Account For better performance.
If not Upgraded your email box account will be suspended now.

Microsoft Verification Team

Do note that your Email box account is about to be suspended if not verify now properly Click Here **<link>** to Verify Now

Microsoft Verification Team

Figure 18: An example of a set of evasive emails

Dear Beneficiary,

My name is ***** ***** and I work with the International Monetary Fund (IMF), I am writing you to let you know that finally your ATM Card worth $650,000.00 USD has been delivered **for activation** via USPS to Mr. ***** ***** who works with the IMF. **Upon activation, your card would be safely delivered** to your home address for use.

Dear Beneficiary,

My name is ***** ***** and I work with the International Monetary Fund (IMF), I am writing you to let you know that finally your ATM Card worth $1,650,000.00 USD has been delivered **through** USPS to Mr. ***** ***** who works with the IMF **where it is going to be activated before final delivery** to your home address.

Dear Beneficiary

Your ATM Visa Card **Will Be Shipped Through** USPS To Your Address I am Mr. ***** *****, **United States Secretary of State by profession. This is to inform you officially that after our investigations with the Federal Bureau of Investigation (FBI), Central Intelligence Agency (CIA) and other Security Agencies in the Country for the year ended 2017 and 2018, we discovered that you have not yet received your over due fund.**

Figure 19: Another example for evasive attempt. We have changed the names due to the confidentiality of the data; however, the names in this set of emails are mostly random, with a few variants as well.

View publication stats